OZONE LOUNGE

POSTER GENERATOR

TECHNICAL DESIGN DOCUMENT

A stage-gated implementation plan for Antigravity

Mobile-first editing | Seven unique daily templates | Premium TTF/OTF fonts | Reliable save and export

Document version: 1.0 - implementation baseline

Document date: 18 October 2026

Product: Standalone Ozone Lounge poster web app

Primary stack: Next.js App Router, TypeScript, Tailwind CSS, PostgreSQL, Prisma

Intended users: Antigravity agent, product owner, implementer and QA reviewer

Delivery method: One bounded stage at a time; verify, report and stop

PROJECT GUARDRAIL

This document is the execution contract. Do not build the entire product in one prompt. The agent must complete only the current stage, run its checks, report exact results and stop. Preserve existing working features and obtain approval before any material architecture or scope change.

DOCUMENT GUIDE

How to use this TDD with Antigravity

Read the product contract and architecture before asking the agent to implement a stage.

Treat this document as a controlled delivery sequence, not permission to implement everything at once. Give Antigravity only the current stage prompt from Part IV, together with the relevant technical sections. When the stage passes, checkpoint the repository and then advance.

Document map

Part I defines product scope, user journeys, source facts and functional requirements.
Part II defines app architecture, document schema, storage, authentication, renderer, fonts and export.
Part III defines the seven distinct template identities.
Part IV contains Stages 0-22, each with a goal, boundaries, tests, acceptance criteria and copy-ready prompt.
Part V and appendices define release verification, operations and reference materials.

GOLDEN RULE

The agent must not skip stages, combine stage prompts or refactor unrelated working code because a rewrite seems cleaner.

The app may use clearly labelled development placeholders during construction. It must not publish invented date, time, venue, phone, social handle or promotional details.

DOCUMENT GUIDE

Contents and navigation

This document is intentionally divided into product rules, architecture contracts, template specifications and stage gates.

Part	Coverage
I. Product contract	Vision, goals, scope, event data, roles, user journeys and requirements.
II. Technical design	Next.js boundaries, registry, state, database, auth, storage, rendering, fonts and export.
III. Template specs	Shared visual contract plus Reddington, Saint, Lannister, Shelby, Lothbrok, Escobar and Pinkman.
IV. Execution stages	Stages 0-22. Each is bounded, testable and includes a prompt and stop condition.
V. Verification and operations	Testing, security, mobile QA, deployment, monitoring and release.
Appendices	Source-of-truth content, example schema, environment checklist, acceptance matrix and references.

How to use a stage prompt

Copy only one stage prompt at a time. Keep the matching acceptance criteria and test list in the same prompt. Do not ask the agent to “continue with all remaining stages.” Require a report of changed files, commands and real results before advancing.

I. PRODUCT CONTRACT

Executive product summary

A single-purpose, standalone creative tool for Ozone Lounge operators.

Ozone Lounge needs a reliable way to produce seven recurring weekly event posters without recreating each design from scratch or depending on a desktop graphic-design workflow. The app is standalone, authenticated and focused on Ozone Lounge marketing posters.

Target outcome

An authorised operator logs in, chooses a named template, edits event details and imagery, previews the result full-screen, then saves an editable draft or downloads a high-quality image.

- Each weekday has a unique creative identity, not simply a different colour scheme.
- App chrome, editor controls, save behavior and export logic remain consistent.
- A saved draft restores content, assets, fonts, palette and layout options.
- Export contains poster artwork only, never editor chrome or zoom controls.
- Template eight should be addable through the registry and a new renderer, not a rewrite.

PRODUCT PROMISE

Fast, repeatable, brand-consistent posters with distinct daily personalities and predictable export quality.

I. PRODUCT CONTRACT

Product goals and success measures

Define success as observable behavior, not subjective completion claims.

Primary goals

Remove repetitive design setup while retaining professional visual quality.

Make the entire workflow practical on a phone, including one-handed access to primary actions.

Keep source-provided event names, performers and offers accurate.

Preserve drafts and durable image references through refreshes and new sessions.

Make the export faithfully reproduce the live poster at exact master dimensions.

Measures to test

A new operator can choose the desired weekday from name, tag and thumbnail alone.

A normal poster can be completed by editing fields and choosing curated style options, without manual positioning of every element.

Successful login, save/reopen, preview/zoom and download flow works on a target phone browser.

All seven defaults pass content regression tests.

A new template can be added without editing duplicated gallery lists, save logic or export naming rules.

I. PRODUCT CONTRACT

Scope and non-goals

Keep the first release focused enough for iterative agent-driven development.

In scope

Username/password login with public registration disabled.
Monday-to-Sunday gallery with seven unique templates.
Template-specific event fields, imagery, font choices, palettes and layout options.
Full-screen preview with pinch zoom, pan, plus/minus, Fit and X to exit.
Draft creation, save, list, reopen, duplicate and delete, subject to the approved visibility rule.
PNG and/or high-quality JPEG export at the configured master dimensions.
Dark app UI by default with an optional bright UI theme.

Out of scope unless separately approved

AI-generated art or copy; social-network publishing; posting calendar; freeform Canva-like design; billing; public self-registration; template marketplace; complex offline collaboration; additional aspect ratios.

SCOPE GUARD

An enhancement is a proposal, not an instruction. Record it and keep the current stage narrow.

I. PRODUCT CONTRACT

Source interpretation and evidence hierarchy

Separate user-provided facts from engineering choices and unresolved decisions.

Authoritative project inputs

The existing Lucky House Poster Generator Architecture is the reference for fixed-canvas rendering, viewport scaling, local preview of images, custom fonts, raster export and saved drafts. The Ozone Lounge weekly outline is the source of truth for its event names, performers, stated offers and visual directions.

Reuse, but do not copy blindly

- Reuse the fixed coordinate canvas, state serialization, asset lifecycle and export adapter concepts.
- Do not bring across safari-specific pricing tables, inclusion lists, excursion cards or testimonial controls unless an Ozone template truly needs them.
- Use one source of truth for Ozone template metadata.
- Technical choices must follow current official documentation for the exact dependency version pinned in the repository.
- If an input conflicts with the source brief, record it instead of silently changing source content.

IMPORTANT

The weekly brief does not supply confirmed start times, venue/address, contact details, official logo file, or a Friday promotion. These must remain configurable and un-invented.

I. PRODUCT CONTRACT

Assumptions and decision register

Development can proceed with reversible defaults; production must close critical unknowns.

Reversible development defaults

Use a configurable 1080 x 1350 px (4:5) canvas as a provisional default only if feed publishing is the likely main use.
Use dark app theme by default.
Seed the seven template definitions and source-provided event copy.
Leave unknown logistics fields blank and render them conditionally.
Use development-only placeholders until approved photos and logo are available.

Decisions that block final production setup

Main channel and aspect ratio: feed 4:5 or story/status 9:16.
Official Ozone Lounge logo and approved image source.
Event start time, venue/location, contact number and social handle.
Whether Friday will later get a special offer.
Internal email/account provisioning constraint for username/password authentication.
Whether drafts are private per operator or shared by the lounge team.

DECISION RULE

Unknown information is a blank/null field plus a visible preflight warning, not invented poster copy.

I. PRODUCT CONTRACT

Users, roles and permissions

Start with a minimal role model and enforce it on the server.

Operator

Can sign in, browse templates, create posters, edit permitted drafts, upload assets, preview and export. Whether operators see only their own drafts or all shared drafts must be decided before production use.

Administrator

Can provision or disable accounts and perform policy-approved draft-management tasks. Admin checks must use server-verified role data; hiding an admin button is not security. Public self-registration is disabled for MVP.

Unauthenticated visitor

Can access login and any intentionally public help content only. They must not list drafts, read private asset metadata, upload media or operate against a stored poster document.

Role	Poster functions	Draft permissions	User administration
Unauthenticated	None	No read/write	None
Operator	Create/edit/preview/export	Own/shared per decision	None
Administrator	All operator functions	Per policy	Provision/disable users

I. PRODUCT CONTRACT

Main user journey and recovery flow

The workflow should be short, predictable and recoverable.

Happy path

1. Login. 2. Choose a weekday card and template name. 3. Edit event/performer/offer details, images and style options. 4. Open full-screen preview, zoom if needed and close with X. 5. Save the editable draft or download the final image.

Recovery path

A failed save keeps the current edits and leaves the document dirty.
A failed upload retains local preview where possible and offers retry.
A failed export shows a useful message and retry action.
A session expiry protects server operations and should preserve a local recovery snapshot where safe.
No operation should silently reset the document to template defaults.

COPY-READY / TECHNICAL BLOCK

```
LOGIN -> GALLERY -> EDITOR -> PREVIEW -> X -> EDITOR
                    |              |
                    +-> SAVE -> DATABASE +-> DOWNLOAD -> PNG/JPEG
```

I. PRODUCT CONTRACT

Information architecture and routes

Keep route semantics clear and protected workspace pages grouped.

Suggested routes

/login - username/password sign-in.
/app/templates - weekly template gallery.
/app/templates/[templateId] - new poster/editor route.
/app/drafts - saved draft library.
/app/drafts/[draftId] - direct draft open or redirect into editor.
/app/settings - application theme and profile/password preferences.
/app/admin/users - optional administrator-only account provisioning.
/api/auth/[...all] - if required by chosen auth library.
/api/drafts and /api/assets or equivalent server actions - chosen based on project conventions.

Navigation rules

Back navigation must preserve edits or prompt before discarding. URLs should carry durable IDs such as templateId and draftId, not every keystroke or palette hover. Every data action checks the session and access policy server-side.

I. PRODUCT CONTRACT

Functional requirements: authentication

A small login screen still needs production-grade session and access handling.

Required behavior

Login fields are username and password only.
Public account creation is disabled unless expressly approved later.
Invalid credentials produce a generic message that does not reveal whether a username exists.
Repeated failures are throttled/rate-limited.
Sessions use secure, HTTP-only cookies with production-appropriate attributes and expiry.
Logout invalidates the session; expiry redirects safely without silently losing edits.

Provisioning constraint

A maintained authentication library is preferred; do not hand-roll password hashing or session security casually. Better Auth documents username sign-in, but its documented sign-up flow still includes email metadata. This must be resolved deliberately: a hidden/internal real email field may be part of account provisioning, or a different maintained auth setup may be needed for a truly email-free account model. Do not invent fake emails as a shortcut.

I. PRODUCT CONTRACT

Functional requirements: template catalogue

Gallery content must be driven from the registry, not duplicated in route components.

Gallery behavior

Each weekday card shows day, unique template name, event title, short creative description, best-for tag and representative thumbnail. The initial order is Monday through Sunday. Search/filter can be deferred until the seven entries are easy to browse.

Registry rules

- A stable ID uniquely identifies each template.
- Display order is explicit and not derived from incidental object order.
- Card metadata and defaults come from one registry definition.
- A template can later be retired from selection without erasing historic draft identity.
- Every thumbnail should depict the actual renderer, not a generic stock card.
- The names are internal creative labels. They do not instruct the implementation to imitate protected fictional characters, performers, show graphics or existing poster layouts.

Acceptance means each card opens the correct renderer with correct defaults.

I. PRODUCT CONTRACT

Functional requirements: poster editing

The MVP is curated composition editing, not arbitrary freeform design.

Shared fields

Event title/day label.
Date and start time.
Venue/location, contact and social handle.
DJ names and host names as separate role fields.
Primary and secondary offer copy where needed.
Background/secondary image slots and crop/focal settings.
Palette and curated typography preset.
Template-specific decoration/layout choices.

Reactive behavior

All controls and the renderer use one canonical document state. Content changes update the live poster without full reload; text input must retain cursor position. Invalid content produces field-specific warnings. Resetting a template requires confirmation or a safe undo path. Each template exposes only its relevant controls, not a generic panel containing irrelevant options.

I. PRODUCT CONTRACT

Functional requirements: drafts and persistence

A saved draft is an editable design recipe, not just a flattened image.

Required operations

Create a new poster from template defaults.
Save a new draft, then update the same record on later saves.
List drafts with template/day, name, updated time and optional thumbnail.
Open and validate a saved draft, preserving its visual styling.
Duplicate into a new ID so the source remains unchanged.
Delete with confirmation and policy-aware authorization.

Persistence integrity

Persist template ID/version, schema version, content, style choices and durable asset references. Store image binaries in object storage, not as huge base64 blobs in JSON. Reject blob: URLs in saved state. Validate client-side for feedback and server-side for integrity. Delete assets only after verifying they are not referenced elsewhere. A network error must not discard the active in-memory document.

I. PRODUCT CONTRACT

Functional requirements: export and download

Export the artwork root, never the scaled viewport or editor page.

Required behavior

Support PNG and/or high-quality JPEG at exact configured master dimensions.
Generate a sanitised file name based on Ozone Lounge, day, template and event title.
Show export progress while resources load and rasterisation runs.
Warn about missing required details and text overflow before export.
Offer a retry path when asset loading or rasterisation fails.

Fidelity contract

Fonts, image crop/focal point, gradients, shadows, SVG masks, logo treatment and line breaks must match the live poster as closely as supported by the chosen engine. Wait for font and image readiness. Preview zoom is transient UI state and must never change export dimensions. Test browser download behavior on actual target phone browsers before declaring launch ready.

I. PRODUCT CONTRACT

Non-functional requirements

Quality is part of the definition of done for every stage.

Reliability

A save status reflects server acknowledgement, not a button click.
A failed save preserves current edits and retry capability.
Export never silently drops custom fonts or image layers.
Blank optional fields hide cleanly and never leave empty panels.
Errors are actionable and do not expose stack traces or secrets.

Usability and performance

Mobile is the primary target; controls have comfortable touch targets and safe-area spacing.
Fit-to-screen preview never crops the canvas.
Heavy rasterisation runs only when requested, not on every keystroke.
Gallery uses lightweight thumbnails; the editor loads only needed assets/fonts where practical.
Current target browsers and lower-memory phones are tested before launch.

Maintainability

Use explicit typed IDs and schema validation, small renderer components, controlled style options and regression fixtures. Add a template via registry, definition, renderer, thumbnail and tests rather than editing unrelated app logic.

II. TECHNICAL DESIGN

Architecture overview

Separate app chrome, editing state, poster rendering, export and persistence.

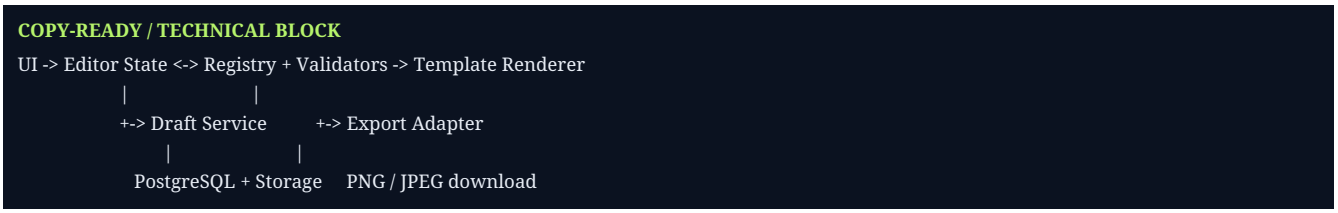
Responsibilities

App shell: navigation, session-aware layout, app theme, general feedback.
Gallery: registry metadata, thumbnails, template selection.
Editor: form controls that update canonical PosterDocument state.
Template renderer: a unique composition for the selected template.
Poster canvas: fixed master coordinate space displayed through a scale wrapper.
Export adapter: waits for assets/fonts and rasterises only the export root.
Draft service: server authorization, validation and durable JSON persistence.
Asset service: upload validation, stable storage reference and metadata.

Boundary rules

Rendering components do not write directly to the database. API/server actions do not reach into browser-only state. Database and storage providers sit behind small service modules. The client receives only the serialisable data it needs.

COPY-READY / TECHNICAL BLOCK



II. TECHNICAL DESIGN

Source tree and module boundaries

Add only the folders needed for the current stage; avoid empty abstraction scaffolding.

Proposed structure

COPY-READY / TECHNICAL BLOCK

```
src/
  app/ (login, protected app, routes/actions)
  components/app-shell/
  components/auth/
  components/gallery/
  components/editor/
  components/poster/PosterCanvas.tsx
  components/poster/templates/{Reddington,Saint,Lannister,Shelby,Lothbrok,Escobar,Pinkman}Template.tsx
  features/{auth,drafts,assets,export,templates}/
  lib/{auth,db,storage,validation}/
  templates/{registry,definitions,palettes,typography}/
  types/poster.ts
public/{fonts,logos,template-thumbnails}/
prisma/schema.prisma
```

Boundaries

App Router routes own navigation and page composition. Feature modules own business operations. templates/registry.ts owns gallery metadata. Individual renderer components own actual visual composition. Server-only modules own secrets, database and storage operations. Do not create every suggested file on day one if the stage does not yet need it.

II. TECHNICAL DESIGN

Next.js server/client boundaries

Protect server resources and keep the interactive client surface purposeful.

Server-side responsibilities

Resolve sessions and authorize access to drafts/assets.
Load validated initial draft data and return serialisable data to the editor.
Validate documents again before durable writes.
Use current official Next.js App Router patterns for the exact pinned version.
Keep database clients, storage service keys and auth secrets in server-only modules.

Client-side responsibilities

Interactive editor, live canvas, file input, preview gestures, zoom/pan and download initiation.
Maintain one canonical in-progress document state.
Dynamically load expensive export code if needed to reduce initial app payload.
Keep transient UI state such as zoom level, open accordion and app theme outside PosterDocument.
Do not rely on middleware/proxy redirects as the only defense; each data operation still checks authorization.

II. TECHNICAL DESIGN

Template registry contract

Registry metadata is the single source of truth for template discovery and initialisation.

Each entry should include stable ID, display name, weekday/order, event title, best-for tag, description, version, thumbnail, defaults, palette presets, typography presets, layout options, editor schema, asset slots and renderer reference.

Registry invariants

IDs are unique and match the fixed TemplateId union.
All default palette/font/layout IDs exist.
Each definition has a valid template version and default PosterDocument.
Gallery ordering is explicit.
Default copy matches the weekly brief.
A test confirms all seven registry entries resolve to a renderer, defaults and thumbnail.

COPY-READY / TECHNICAL BLOCK

```
type TemplateId = "reddington" | "saint" | "lannister" | "shelby" | "lothbrok" | "escobar" | "pinkman";
interface TemplateDefinition { id: TemplateId; name: string; dayOfWeek: number; eventTitle: string; tag: string; templateVersion: number; defaults: PosterDocument; editorSchema: TemplateEditorSchema; Renderer: React.ComponentType<TemplateRendererProps>; }
```

II. TECHNICAL DESIGN

Canonical PosterDocument schema

Persist every design choice required to reconstruct a poster accurately.

Document fields

schemaVersion: document structure version.
templateId and templateVersion: select renderer and compatibility policy.
canvas: width, height and named format identifier.
content: event title, date/time, venue/location/contact, performer roles, offers and custom lines.
assets: durable asset IDs/references for logo, hero and optional secondary images.
styling: palette ID, typography preset, layout ID, decorations and focal points.
metadata: draft name and timestamps where appropriate.

Never persist these as design fields

Zoom/pan, open accordions, focus/hover, interface theme, blob: URLs, local file objects, raw image binaries or arbitrary CSS code. Keep style option identifiers constrained by the registry so malformed JSON cannot inject unsupported layout behavior.

II. TECHNICAL DESIGN

Database models and relationships

Use PostgreSQL for durable metadata and JSON; object storage for binary media.

User and auth tables

The auth library owns its core account/session tables. Add username/role/status fields only through documented extension points and migrations; do not casually rewrite auth-owned schema.

PosterDraft

Expected fields: UUID/ID, owner or creator ID, display name, template ID/version, schema version, JSON document, optional thumbnail asset ID, created and updated timestamps. Index owner, update time and template for list operations.

MediaAsset

Expected metadata: stable asset ID, storage key, original filename, detected MIME type, byte size, dimensions, uploader and created time. The binary itself stays in object storage.

AuditEvent (optional but useful)

Record actor, action, target record, timestamp and safe metadata for login, save, delete or account administration. Do not log secrets or all poster text by default.

Entity	Storage	Purpose
User/auth	PostgreSQL	Identity/session metadata
PosterDraft	PostgreSQL	Editable poster recipe
MediaAsset	Metadata in DB, bytes in object storage	Durable images
AuditEvent	PostgreSQL if enabled	Key action traceability

II. TECHNICAL DESIGN

Draft service and API contracts

Implement business rules behind one server-side service boundary.

Draft operations

List with safe filters and stable ordering; create from validated content; get one with access check; update a known ID; duplicate into a new ID; delete only under approved access policy. Each returns a predictable result/error contract that the UI can display.

Asset operations

Upload verifies user, bytes, allowed type and size. It returns stable asset ID and safe metadata. Deletion/replacement follows reference checks. Avoid arbitrary remote URL fetching from the server because it can introduce SSRF. Do not expose storage service keys to browser bundles.

Error contract

Return structured error codes for validation, unauthorised, not found, conflict, asset failure and server failure. Hide stack traces from operators. Preserve local state when requests fail.

COPY-READY / TECHNICAL BLOCK

```
type ServiceResult<T> = { ok: true; data: T } | { ok: false; error: { code: string; message: string } };
```

II. TECHNICAL DESIGN

Editor state and update model

One source of truth prevents the controls and poster preview from drifting apart.

Recommended initial approach

Use a React reducer or small scoped store first. Do not add a global state dependency unless the component tree proves it is needed. Actions should describe domain changes: set content field, replace/remove asset, select palette, select typography, select layout, update focal point or reset to defaults.

State rules

Control values are derived from PosterDocument rather than duplicated in unrelated local states.
Updating one field preserves all unrelated nested fields.
Template-specific options are validated against the active schema.
Zoom and pan are preview component state, never poster design state.
Theme preference is independent of poster palette.
Switching to a new template must save/confirm before replacing unsaved changes.

Performance

Keep typing responsive, memoise complex renderer fragments only when measurement justifies it, and avoid export work on every keystroke. Autosave is added only after manual persistence is trustworthy.

II. TECHNICAL DESIGN

Validation and fallback rules

Missing optional values should remove components cleanly; invalid values should be visible.

Input validation

Check template IDs, schema versions, canvas format, palette/font/layout IDs, focal point bounds and string length limits.

Validate on the client for feedback and on the server for integrity.

Verify asset IDs belong to the current user/shared workspace policy.

Reject unknown decoration properties instead of passing arbitrary CSS through state.

Layout fallbacks

Blank offer: hide offer component and rebalance the composition.

Blank venue/time/date/contact: hide the associated component but include a preflight warning where configured required.

Missing photo: use a deliberately designed approved fallback or show a development-only placeholder; do not export a broken image icon.

Long title/DJ/offer: use a tested wrap/layout option or ask the operator to shorten copy. Do not shrink everything below a template minimum.

Missing font: show warning and deliberate fallback; do not silently claim output fidelity.

II. TECHNICAL DESIGN

Master canvas and output dimensions

A responsive viewport displays the canvas; it never changes the design coordinate system.

Use a fixed master canvas and define all positions, font sizes, margins, masks and shadows within it. The blueprint recommends a configurable 1080 x 1350 px, 4:5 first format if feed placement is confirmed. The source brief does not name the publishing channel, so this is provisional.

Invariants

The master width/height live in one configuration object.
The editor preview scales only visually; export uses unscaled master dimensions.
Each template defines safe margins and text boxes.
Output dimensions are checked by inspecting the actual binary, not just CSS size.
A later 1080 x 1920 story/status format must have deliberate composition rules; do not stretch a feed poster.

PRODUCTION GATE

Confirm the publishing channel before locking the output format across every renderer.

II. TECHNICAL DESIGN

Responsive scaling algorithm

Use a two-tier wrapper because CSS transform does not change the element's layout footprint.

Measure available viewport width and height, subtract safe padding and compute the smaller ratio. Use a ResizeObserver for container changes and handle orientation shifts. Clamp available sizes to a positive minimum.

COPY-READY / TECHNICAL BLOCK

```
const scale = Math.min((availableWidth - padding * 2) / masterWidth, (availableHeight - padding * 2) / masterHeight);
const displayWidth = masterWidth * scale;
const displayHeight = masterHeight * scale;
```

Two-tier wrapper

Outer layout box reserves masterWidth * scale by masterHeight * scale.
Inner poster root remains at master width and height and receives the scale transform.
Overlay controls remain outside the transform so they stay readable.
Preview mode measures the full viewport separately from the editor.
Export targets the unscaled root regardless of current screen scale or zoom.

II. TECHNICAL DESIGN

Layer model and z-index contract

Template renderers should use an explicit layer vocabulary and order.

Recommended conceptual order: background; texture/pattern/glow; secondary image; divider/frame; logo and contrast backing; title/day label; performer/host text; offer blocks; practical details. Numeric z-index values are implementation details but should be centralised or documented per renderer.

Rules

Every template can use fewer layers but should document unique ordering where elements intentionally overlap.
Glows and decorative layers must not intercept controls or obscure text.
The logo remains an independent layer with preserved aspect ratio.
Test CSS filters, blend modes and SVG masks through the real export adapter.
Keep decorative SVGs viewBox-defined and scale predictably.
Avoid hidden editor handles inside the export root.

II. TECHNICAL DESIGN

Typography roles and curated font pairs

Font options should be curated so a user can change style without destroying hierarchy.

Font roles

Display: main event title and visual signature.
Heading/label: day, DJ/host labels and offer headers.
Body: venue, date, contact and practical copy.
Numeral/utility: times or short data that must scan clearly.
Optional accent: only for a template that can support it without reducing readability.

Pair presets

Each template provides allowed font-pair presets mapping font IDs and styles to roles. Target at least two tested presets per template if the actual licensed font inventory supports them. Offer human-readable family labels, not filenames. Set minimum sizes, maximum lines and overflow warnings. The editor's app UI font system is separate from the poster font system.

II. TECHNICAL DESIGN

TTF/OTF font registration and licensing

Supplied TTF/OTF files need a manifest and a readiness check for export.

Font manifest

Store internal ID, family name, source path, file format, weight, style, allowed role(s) and license note. Register with @font-face and stable CSS family names. Encode paths safely where names include spaces. Do not assume a regular file includes bold/italic variants; verify the actual inventory.

Export readiness

- Wait for document.fonts.ready and check required font faces before rasterising.
- Test the bundled font in the actual title and offer copy; a font loading successfully does not guarantee acceptable line breaks.
- Preload only critical faces if startup performance remains acceptable.
- Use a known-good fallback and warn on failed font loading.
- Track rights to redistribute/serve each font in the app; having the file is not proof of web embedding permission.

II. TECHNICAL DESIGN

Image assets and media policy

Photos, textures and logo assets must be durable, safe and approved.

Initial slots

Official Ozone Lounge logo, maintained as a separate layer.
Background/hero photo per template.
Optional secondary/detail image or texture where the template supports it.
Gallery thumbnail generated from approved render or design preview.

Upload policy

Allowlist raster formats such as JPEG, PNG and WebP.
Treat SVG as unsupported unless sanitisation and export security are explicitly addressed.
Validate file size, decoded dimensions and file signature/type server-side where feasible.
Use temporary object URLs for immediate preview and revoke them on replace/remove/unmount.
Store image bytes in object storage and metadata/IDs in PostgreSQL.
Use original or licensed imagery; do not use a template name as a license to copy existing entertainment IP.

II. TECHNICAL DESIGN

Image crop, focal points and fallback behavior

Images should fill their slot without stretching or accidentally cutting off the subject.

Store focal point as normalised coordinates (0..1), not CSS pixels. Resolve crop against the actual slot aspect ratio using cover/object-position or equivalent. A default focal point may be centered, but a template or operator can adjust it.

Fallback behavior

If an image is optional and absent, hide that layer or use a designed background fallback.
If the image is required for launch, block final export or require explicit override based on product policy.
A failed image load must never become a broken image icon in the downloaded poster.
Replacing the image updates the preview immediately and preserves the rest of the document.
Crop adjustment is design state; pinch zoom in the preview is not.

COPY-READY / TECHNICAL BLOCK

```
interface FocalPoint { x: number; y: number } // values within 0..1
```

II. TECHNICAL DESIGN

Asset persistence lifecycle

Bridge instant local feedback with durable saved references.

1. Select a file and validate type/size for immediate feedback. 2. Create a temporary object URL for the editor preview. 3. When saving, upload the binary and receive a permanent asset ID/reference. 4. Replace the transient local reference with the durable reference. 5. Persist the poster document only when referenced assets are durable. 6. Revoke temporary object URLs when no longer needed.

Failure recovery

If upload fails, preserve current edits and local preview, show the failure and offer retry. If an upload succeeds but the database write fails, apply an orphan-asset cleanup or tracking strategy. Never delete an asset still referenced by another draft. On refresh/relogin, saved posters must not rely on a blob: URL.

II. TECHNICAL DESIGN

Export service boundary

Keep the rasterisation implementation replaceable behind a small adapter.

Export contract

Input includes export root, configured dimensions, PNG/JPEG choice, quality and timeout policy. Preflight checks fonts, images, text overflow and missing required logistics. The export result includes Blob, filename, width, height and MIME type. Download starts only after a successful render; temporary download URLs are revoked safely.

Why isolate the exporter

DOM-to-image tools serialize browser layout and may not support every CSS feature identically. One adapter lets the project change library or use a server renderer if needed without rewriting editor logic. Do not couple UI state to a package-specific return type. Measure memory and time on mobile before adding 2x exports. Prevent concurrent exports from overloading memory.

COPY-READY / TECHNICAL BLOCK

```
type ExportFormat = "png" | "jpeg";
type ExportResult = { blob: Blob; fileName: string; width: number; height: number; mimeType: string };
```

II. TECHNICAL DESIGN

Cross-origin images and canvas safety

An image can display in the DOM yet still block canvas export.

A remote asset without appropriate CORS response headers may taint the canvas. Browser APIs can then reject toBlob() or toDataURL(). Do not assume setting crossOrigin is enough; the serving origin must permit the request.

Required controls

Use controlled storage URLs and configure only the application origins required for export.
Prefer same-origin or approved object-storage asset URLs.
Do not build an unrestricted server-side URL proxy to work around CORS; it can create SSRF exposure.
Classify export errors so image/CORS failures differ from generic rasterisation failures.
Test cold-cache assets, changed assets, failed assets and real mobile browsers with the chosen storage provider.

II. TECHNICAL DESIGN

Draft save, reopen and duplication

Build reliable manual save before adding autosave convenience.

Manual save

Validate document, upload pending assets, replace temporary references, create/update the same draft ID, wait for server acknowledgement and then show Saved. On failure keep the document dirty and offer retry. The URL may include a stable draft ID but should not remount the editor unnecessarily.

Open, duplicate and delete

Open: load, validate version, migrate or reject deliberately, hydrate asset references.
Duplicate: clone the document to a new ID, leaving the source untouched.
Delete: confirm intent and enforce owner/shared access server-side.
List: show template/day, title, updated timestamp and optional thumbnail.
Asset deletion: check references before removing a binary.
Empty state: tell the operator how to create the first poster.

II. TECHNICAL DESIGN

Autosave and concurrent editing

Autosave is deferred until manual persistence is trustworthy.

Safe autosave rules

Debounce saves to avoid a database write on every keypress.
Show Saving, Saved, Unsaved and Save failed states explicitly.
Do not autosave until image references are durable and validation passes.
Track a version or updatedAt value to detect overwriting a newer server revision.
A conflict retains current edits and provides reload/duplicate choices rather than silently merging.

MVP recommendation

Manual save is the launch baseline; autosave can follow after testing. Offline sync and conflict-free merge are not MVP features. A failed autosave must never mark the document clean, and a fresh editor session must not erase unsaved text after a transient request failure.

II. TECHNICAL DESIGN

Authentication library decision and caveat

The UI requirement is username/password, but account creation internals still matter.

Use a maintained auth library with server-managed sessions and password hashing. Better Auth documents username-based sign-in through its username plugin. Its documented sign-up flow nevertheless asks for email, name, password and username. This is an integration constraint, not a reason to quietly build custom auth.

Required resolution

Keep the visible sign-in form to username and password only.

Disable public sign-up for MVP.

Decide whether provisioning uses a real internal email field, or select another maintained setup after a small proof of concept.

Never invent fake delivery addresses or store plaintext passwords.

Confirm cookie/session configuration, rate limits and account recovery before production.

Pin the library version and check the matching official docs before implementation.

II. TECHNICAL DESIGN

Authorization and session safety

Authentication establishes identity; authorization checks each requested resource.

Every draft list/get/update/delete and every asset upload/resolve action must verify the current session and permitted access. A hidden navigation item does not secure an endpoint. Admin actions require a server-verified role. Validate IDs and reject malformed inputs before queries.

Session lifecycle

Cookies are Secure in production, HttpOnly and assigned suitable SameSite/expiry values.
Logout invalidates the active session where supported.
Repeated failures are rate-limited and error text avoids account enumeration.
Session expiry during editing requires reauthentication before server writes.
Do not log passwords, tokens, cookies, service keys or entire environment values.
Protect uploaded media with storage access policies or narrowly scoped signed URLs.

II. TECHNICAL DESIGN

Security threat model

Security review spans authentication, data access, uploads, fonts and export.

Main threats

Unauthorized draft access by changing a draft ID.
Oversized, corrupted or MIME-spoofed image uploads.
SVG/script injection or unsafe HTML rendering of poster copy.
Leaked database/storage credentials in client bundles or logs.
Arbitrary remote URL fetching that reaches internal resources.
Unlicensed fonts or unapproved imagery.

Controls

Server-side auth and object-level permission checks for every data action.
Allowlists and size/dimension limits for media, enforced on the server.
Render user copy as text, not injected HTML; treat SVG as unsupported until sanitised.
Server-only environment variables, no private NEXT_PUBLIC_ secrets.
Narrow storage access rules and allowed CORS origins.
Review dependency advisories and run available audit/secret scan checks before launch.

II. TECHNICAL DESIGN

Error states and recovery language

Tell the operator what happened and the next safe action.

Required states

Loading template/draft; unsaved changes; saving/saved/save failed; uploading/rejected image; font/image failed; text overflow; missing publishing detail; exporting/exported/export failed; session expired; empty draft library.

Copy and safety rules

Prefer “The draft was not saved. Your edits are still here. Retry.” over vague errors.
Show Saved only after server acknowledgement.
Never show SQL errors, tokens, storage credentials or stack traces to operators.
A failed export must not mutate or clear the active poster.
A request/correlation ID may help support without logging sensitive content.
Provide non-colour status cues and clear labels for important actions.

II. TECHNICAL DESIGN

Accessibility and touch ergonomics

Pinch zoom is useful but must not be the only available way to inspect the poster.

Input and focus

Large, well-spaced mobile touch targets and safe-area-aware sticky actions.
Keyboard navigation and visible focus on desktop.
Escape closes preview when a keyboard is available.
Preview manages focus and returns it to the trigger when closed.
Plus/minus and Fit controls supplement pinch gestures.
Respect reduced-motion preferences.

Readability

Meet sensible contrast for UI labels in dark and bright themes.
Give icon-only X/zoom controls accessible labels.
Do not rely only on colour for save status or validation.
Keep event title, DJ/host lineup and offer readable at intended phone viewing size.
Interface theme never recolours poster artwork.

II. TECHNICAL DESIGN

Performance and resource budgets

Phone devices have less tolerance for repeated high-resolution image work.

Initial performance rules

- Gallery uses lightweight thumbnails rather than all full-size assets.
- Load only the font families/weights a selected template needs where practical.
- Keep editor typing responsive and avoid rasterisation on every input event.
- Prevent duplicate exports and release object URLs/listeners/references when safe.
- Profile export duration and memory on real devices before considering 2x output.

Operational measurements

Measure export success/failure by category, asset upload failure/size, slow database operations and rerender hotspots. Avoid logging full poster copy or private asset URLs unnecessarily. Use observed performance rather than presenting arbitrary response-time claims as proven facts.

II. TECHNICAL DESIGN

Testing strategy

Every stage needs evidence appropriate to the feature boundary.

Unit tests

Registry completeness/uniqueness; event default values; schema validation; filename sanitisation; scaling math; focal-point bounds; version compatibility and missing-detail warnings.

Component/integration tests

Field updates change the same document used by the renderer; app theme does not change poster colours; preview open/close preserves state and focus; failed save leaves document dirty; protected actions deny unauthorised users.

Browser/E2E tests

Login -> gallery -> editor -> preview -> close -> save -> reopen -> download. Verify image dimensions/MIME type, touch preview, rotate/orient, font loading, blank offers, long names, missing optional image and failed network paths. Use actual devices for browser behavior that emulation cannot prove.

II. TECHNICAL DESIGN

Visual QA and export parity

Editor screenshots are not a substitute for inspecting generated binaries.

Reference fixtures

Build known tests for normal title/performer, long title, long DJ/host names, multi-line offers, blank offer, missing logistics, light/dark images and every supplied font family/weight.

Side-by-side checks

Font family, weight, line breaks and baselines.
Image crop and focal point.
SVG masks, edge dividers and alignment.
Glow, gradients, shadows, opacity and blend modes.
Logo contrast and position.
Exact binary dimensions, MIME type and sensible file size.

REGRESSION RULE

Once accepted, retain a deterministic fixture or visual baseline per template so shared-engine changes cannot silently degrade it.

II. TECHNICAL DESIGN

Shared visual contract across templates

Creative difference and brand consistency must coexist.

Required differences

Distinct composition and title geometry.
Different image framing and decorative vocabulary.
Different treatment of DJ/host information and promotions.
Coordinated, named palette choices and curated font pairs.
Unique thumbnails that reflect each renderer.

Required consistency

Official logo remains recognisable and legible.
Day/event title is prominent, with performer lineup and offer next.
Practical details appear in a safe region when supplied.
Safe margins and output dimensions remain consistent.
The same save/preview/export behavior applies to every template.
Empty fields are omitted deliberately; no invented offer percentage or logistics copy.

III. TEMPLATE DESIGN SPECS

Reddington - Monday Sundowners

Event: Old Classics & Sundowners | Tag: Best for Monday Sundowners

Source facts

DJ/host: Monsieur Pierre.
Offer: Discounts on all drinks, excluding wine.
Visual direction: retro classics, sunset colours and vintage music aesthetics.

Proposed composition

Warm full-bleed hero; prominent retro title; subtle sun-disc/vinyl anchor; compact DJ block; ticket/ribbon-style offer panel; subtle grain. Palette candidates: Sunset Ember, Golden Hour and Vinyl Noir. Pair an expressive retro display face with compact labels and a readable body font.

Acceptance details

The exclusion “excluding wine” must remain visible. Blank date/venue fields hide cleanly. Long event title follows a tested wrap rule. Palette changes do not change app theme. Reddington must have its own renderer and export fixture; it cannot be a shared generic layout with only new colours.

III. TEMPLATE DESIGN SPECS

Saint - Tuesday African Night

Event: African Night (Afro) | Tag: Best for Afrobeat Nights

Source facts

Line-up: DJ Neb & DVJ Jojez.
Offer: Discounts on beer buckets and cocktails.
Visual direction: African-inspired patterns, Afrobeat energy and bold colours.

Proposed composition

Asymmetrical geometry, strong title block and a hero image framed by original repeating shapes/lines. Palette candidates: Saffron & Ink, Electric Azure and Terracotta Pulse. Use bold display type paired with clear labels/body copy. Pattern work should be contemporary and original rather than stereotyped clip art.

Acceptance details

The design remains recognisable with no secondary image. Pattern density and colour options cannot overlap the text-safe area. Offer copy remains complete. Layout differs structurally from Reddington, not merely in colour, and the gallery thumbnail reflects the real renderer.

III. TEMPLATE DESIGN SPECS

Lannister - Wednesday Groove

Event: Groovy Wednesday | Tag: Best for Midweek Groove

Source facts

DJs: DJ Donben & DJ Vin.
Primary offer: Free koroga chicken on the house.
Secondary offer: Discounts on whiskey bottles, cognac and beer buckets.
Visual direction: groovy disco aesthetics, neon lights and dance-floor energy.

Proposed composition

Oversized title; disco-ball/spotlight focal element; neon edge lights; separated DJ panel; high-contrast offer card. Palette candidates: Ultraviolet Club, Lime & Magenta and Cyan After Dark. Offer hierarchy is important: free koroga chicken must read first; discounts remain a secondary grouping.

Acceptance details

Glow stays behind the text and must export reliably. Blank secondary offer hides cleanly. Test strong/medium/low glow presets and confirm the composition stays legible on a real phone. Do not collapse both offers into a dense paragraph.

III. TEMPLATE DESIGN SPECS

Shelby - Thursday Karaoke Night

Event: Karaoke Night | Tag: Best for Karaoke Night

Source facts

Hosts: Bellah Harriet & Tito Wagithomo.
DJ: DJ Fab.
Primary offer: Free tequila shots for singers.
Secondary offer: Discounts on gin, rum and vodka.
Visual direction: microphones, stage spotlights and a lively sing-along atmosphere.

Proposed composition

Stage-view image, microphone accent and spotlight cone; strong karaoke headline; hosts and DJ separated by role; primary singer offer visually distinct from spirit discounts. Palette candidates: Stage Crimson, Midnight Violet and Spotlight Gold.

Acceptance details

Keep host names in a hosts field and DJ Fab in a DJ field. “For singers” must remain unmistakable. Test long names, optional secondary image removed and raster export with spotlight geometry. Do not combine roles into one difficult-to-scan string.

III. TEMPLATE DESIGN SPECS

Lothbrok - Friday Flight Fridays

Event: Flight Fridays | Tag: Best for Premium Fridays

Source facts

DJs: DJ Slim D & Mistabrand.
No special offer was supplied in the original brief.
Visual direction: premium nightlife, energetic DJ visuals and a high-energy Friday atmosphere.

Proposed composition

Hero DJ/nightlife photo; diagonal motion or premium split composition; confident title; restrained metallic highlights. Palette candidates: Black & Chrome, Ice Blue and Aubergine Luxe. Use a composition that feels premium and energetic, but less decorative than Escobar and less neon-heavy than Lannister.

Acceptance details

Leave offer fields blank. Do not print “No special offer supplied.” Omit the offer component and rebalance the layout so there is no empty card or awkward hole. Provide a test variant where a future user-entered offer is present.

III. TEMPLATE DESIGN SPECS

Escobar - Saturday Fiesta

Event: Saturday Fiesta | Tag: Best for Saturday Fiesta

Source facts

DJ: DVJ Jojez.
Offer: Discounts on whiskey bottles and cognac.
Visual direction: fiesta energy, colourful lights and vibrant weekend celebration.

Proposed composition

Layered colour blocks or light orbs, celebratory hero crop, bold weekend title and compact offer treatment. Palette candidates: Fiesta Citrus, Coral Electric and Purple & Gold. Use asymmetry and layered light rather than copying Saint's geometry or Lannister's neon disco language.

Acceptance details

The title and offer remain legible across bright and dark images. Removing optional texture does not break composition. Palette changes update coordinated tokens, not just one accent. Confirm the renderer is distinct during visual QA and include an export regression fixture.

III. TEMPLATE DESIGN SPECS

Pinkman - Sunday Old Skool

Event: Old Skool Sundays | Tag: Best for Old Skool Sundays

Source facts

DJ: DJ Ngash.
Offer: Discounts on beer buckets.
Visual direction: throwback music, vinyl records, nostalgic classics and relaxed Sunday vibes.

Proposed composition

Nostalgic background texture, vinyl/cassette motif, warm palette and relaxed title hierarchy. Palette candidates: Amber Memory, Faded Teal and Brick & Cream. Use a clear lower information region and keep decorative texture away from practical copy.

Acceptance details

The design must not look like a recoloured Saturday Fiesta or Monday sundowner. Motif/texture can be reduced or disabled if the selected photo already supplies that language. Check offer legibility, crop and full-size export with texture both enabled and disabled.

III. TEMPLATE DESIGN SPECS

Template eight and later: extension contract

Extensibility is proven only when adding a template is a bounded, testable task.

A new template must include stable ID/name/day/tag, a preview thumbnail, original renderer, versioned defaults, template-specific editor schema, palette and typography presets, asset slots/fallbacks, and tests.

Extension gate

Registry uniqueness and default-document validation pass.
Gallery card is driven by registry metadata.
Editor controls are generated/selected through the template schema.
Long copy, blank offers and missing images are tested.
Save/reopen preserves all styling choices.
Export dimensions and visual fidelity pass.
Existing historic drafts remain renderable or are deliberately migrated.
No change to shared rendering/editor logic unless a genuine shared capability is needed.

III. TEMPLATE DESIGN SPECS

Brand assets and logo placement

The Ozone outline requires the official lounge logo and consistent placement.

Asset readiness

Obtain an approved transparent logo and approved event/DJ imagery. Keep the logo as a separate layer so templates can position it without baking it into photographs. Preserve aspect ratio and use contrast treatment only where needed.

Brand integrity

Do not recolour or distort the official logo unless an approved variant is supplied.
Use a safe placement zone and consistent treatment across all seven designs.
Validate readability over both dark and bright backgrounds.
Use original or licensed images and decorative artwork.
Record font and image licences in the asset inventory.
The project source has not yet provided the final logo/approved photo library; production asset readiness remains a gate.

III. TEMPLATE DESIGN SPECS

Publishing information and export preflight

The source brief explicitly asks for event logistics to be confirmed before publishing.

Editable fields

Date/date text, start time, venue name, location/address, contact number, social handle and any additional entry guidance only if Ozone approves the wording.

Preflight rules

Drafts may be saved incomplete.
Export shows a checklist of missing fields configured as required.
The user may proceed with an explicit override if the owner chooses that policy.
Empty optional fields are hidden from the artwork rather than printed as placeholders.
The app never makes up a real-world time, venue or phone number.
Friday's blank offer is valid source data, not a validation error.

PRODUCTION GATE

Confirm the official logo, approved imagery, publishing channel and required event logistics before final launch.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 00 - Repository reconnaissance and requirements lock

Inspect before changing anything; establish a verified baseline.

Objective

Inspect before changing anything; establish a verified baseline.

Preconditions

- Read this TDD's applicable product, architecture and template sections before editing.
- Confirm previous stages that this stage depends on are accepted; otherwise report the blocker and stop.
- Inspect the live repository and fit implementation to actual code rather than assuming the example paths are exact.

Work items - do only these

- Read the existing repository, package.json, lockfile, routing, CSS system, auth/database setup and tests before proposing changes.
- Compare repository state with this TDD and identify implemented, partially implemented and missing areas.
- Confirm the selected runtime, package manager, deployment target, database provider, storage provider and existing conventions from available evidence.
- Create or update a concise decision register for unresolved choices: primary poster channel/aspect ratio, username provisioning model, draft visibility, approved logo/images, event logistics and Friday offer.
- Record existing commands for install, lint, type-check, unit tests, integration tests, build and migrations. Do not claim commands pass unless executed.
- Create an implementation log with current commit/hash if available, repository state, decisions, risks and next safe step.

Files and modules in scope

- Documentation-only stage; no application source changes unless a clearly broken project instruction must be corrected and approved.
- Suggested outputs: docs/implementation-log.md, docs/decision-register.md, docs/repo-baseline.md.
- Identify which TDD sections apply directly and which decisions must stay provisional.

BOUNDARY

Do not install dependencies, rewrite configuration or begin implementation. Stop after reporting repository facts, existing defects, decisions and a recommended stage order.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 00 - Acceptance and Antigravity prompt

The stage is complete only when its evidence is reported and its gate is passed.

Acceptance criteria

The agent reports the actual stack and project state from files, not assumptions.
All blocking questions are recorded with impact and a recommended reversible default.
Baseline commands and current failures are reported accurately.
No working feature, configuration or user data has been changed.
A stage-by-stage execution order is acknowledged.

Verification plan

Inspect repository and package manifests; summarize current routes, data/auth/storage and tests.
Run only safe read-only baseline commands and the existing test/build commands if dependencies are present.
Review the TDD decision list and produce a brief risk report.
Owner reviews the unresolved production decisions before release stages.

STOP CONDITION

Do not install dependencies, rewrite configuration or begin implementation. Stop after reporting repository facts, existing defects, decisions and a recommended stage order. Do not proceed to the next stage until the owner accepts this report.

Copy-ready Antigravity prompt

COPY-READY / TECHNICAL BLOCK

You are implementing Ozone Lounge Poster Generator TDD Stage 00: Repository reconnaissance and requirements lock. Implement only this stage. First inspect the current repository and the applicable TDD sections. Preserve working code and completed stage behavior. Stay within the listed scope; do not begin later stages or unrelated refactors.

Use current official documentation for the exact dependency versions in the lockfile. Validate all untrusted inputs and keep secrets server-only. Add/adjust focused tests for this stage. Run relevant tests, type-check, lint and build where feasible. Never claim a check passed unless you ran it and report its actual result.

At completion, report: (1) changed files, (2) implementation decisions and assumptions, (3) commands run and pass/fail results, (4) acceptance criteria passed/failed/not tested, (5) risks or owner decisions, (6) next stage only if this gate passes. If blocked, report the minimal blocker and stop without guessing.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 01 - Project bootstrap and dependency baseline

Create a stable development baseline that Antigravity can build on safely.

Objective

Create a stable development baseline that Antigravity can build on safely.

Preconditions

Read this TDD's applicable product, architecture and template sections before editing.

Confirm previous stages that this stage depends on are accepted; otherwise report the blocker and stop.

Inspect the live repository and fit implementation to actual code rather than assuming the example paths are exact.

Work items - do only these

If the repository is empty, scaffold a Next.js App Router TypeScript project using the chosen package manager; if it already exists, preserve its structure and avoid replacement scaffolding.

Pin compatible dependency versions and commit/update the lockfile. Confirm versions from current official documentation before selecting APIs.

Configure strict TypeScript, linting, formatting and reliable local development/build commands.

Add a minimal application entry point and health-checkable home route without designing the full experience.

Add .gitignore protection for environment files, build output and local generated assets.

Create an example environment file containing variable names and placeholders only.

Files and modules in scope

package.json and lockfile; next.config.*; tsconfig.json; eslint config; .gitignore; .env.example.

Initial app route/layout and a short README with setup/test commands.

Do not configure production secrets in the repository.

BOUNDARY

Implement only the project bootstrap. Do not create authentication, database models, templates or editor behavior. Record exact dependency versions and stop after tests pass.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 01 - Acceptance and Antigravity prompt

The stage is complete only when its evidence is reported and its gate is passed.

Acceptance criteria

A clean dependency install succeeds with the recorded package manager.
Type check, lint and production build pass or known baseline defects are explicitly logged.
App starts locally and renders the minimal route.
No secret values are committed or exposed to client bundles.

Verification plan

Run clean install from lockfile.
Run lint, type-check and production build.
Start the app and verify the route in a browser.
Inspect git diff for unrelated or destructive changes.

STOP CONDITION

Implement only the project bootstrap. Do not create authentication, database models, templates or editor behavior. Record exact dependency versions and stop after tests pass. Do not proceed to the next stage until the owner accepts this report.

Copy-ready Antigravity prompt

COPY-READY / TECHNICAL BLOCK

You are implementing Ozone Lounge Poster Generator TDD Stage 01: Project bootstrap and dependency baseline. Implement only this stage. First inspect the current repository and the applicable TDD sections. Preserve working code and completed stage behavior. Stay within the listed scope; do not begin later stages or unrelated refactors.

Use current official documentation for the exact dependency versions in the lockfile. Validate all untrusted inputs and keep secrets server-only. Add/adjust focused tests for this stage. Run relevant tests, type-check, lint and build where feasible. Never claim a check passed unless you ran it and report its actual result.

At completion, report: (1) changed files, (2) implementation decisions and assumptions, (3) commands run and pass/fail results, (4) acceptance criteria passed/failed/not tested, (5) risks or owner decisions, (6) next stage only if this gate passes. If blocked, report the minimal blocker and stop without guessing.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 02 - Database and persistence foundation

Establish a safe, migration-driven persistence layer before auth and drafts depend on it.

Objective

Establish a safe, migration-driven persistence layer before auth and drafts depend on it.

Preconditions

Read this TDD's applicable product, architecture and template sections before editing.

Confirm previous stages that this stage depends on are accepted; otherwise report the blocker and stop.

Inspect the live repository and fit implementation to actual code rather than assuming the example paths are exact.

Work items - do only these

Use PostgreSQL with Prisma only if confirmed in the repository/decision log; do not silently switch providers.

Check Prisma setup instructions for the exact pinned major version and configure server-only database access.

Define a minimal initial schema for accounts/auth adapter compatibility if required, users/roles as required by chosen auth, poster drafts and media metadata; do not over-model future features.

Create explicit migrations and seed strategy; seeds must use source-backed weekly defaults and no secrets.

Define indexes, created/updated timestamps, ownership or workspace scope fields, and a future-safe template ID/version representation.

Keep JSON poster document validation in application code; database JSON does not replace schema validation.

Document migration, seed and reset commands with safety warnings for destructive operations.

Files and modules in scope

prisma/schema.prisma and migrations (or the confirmed equivalent schema/migration location).

Server-only database client module and repository/service pattern.

Seed data for non-sensitive app defaults only; test database instructions.

BOUNDARY

Implement only the persistence foundation. Do not implement login UI, editor UI or poster rendering. If the provider or auth schema assumptions are unresolved, isolate the decision and stop rather than guessing.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 02 - Acceptance and Antigravity prompt

The stage is complete only when its evidence is reported and its gate is passed.

Acceptance criteria

Migration applies on a clean development database and schema status is correct.
Database code is never imported into browser-only modules.
Queries anticipate owner/workspace boundaries and are not globally readable by default.
Seed can be re-run safely or clearly documents a one-time behavior.

Verification plan

Run Prisma validation/generate using the pinned version.
Apply migration to a disposable empty database and exercise a minimal create/read/update/delete test.
Run type-check and build.
Verify no production data is at risk from reset scripts.

STOP CONDITION

Implement only the persistence foundation. Do not implement login UI, editor UI or poster rendering. If the provider or auth schema assumptions are unresolved, isolate the decision and stop rather than guessing. Do not proceed to the next stage until the owner accepts this report.

Copy-ready Antigravity prompt

COPY-READY / TECHNICAL BLOCK

You are implementing Ozone Lounge Poster Generator TDD Stage 02: Database and persistence foundation. Implement only this stage. First inspect the current repository and the applicable TDD sections. Preserve working code and completed stage behavior. Stay within the listed scope; do not begin later stages or unrelated refactors.

Use current official documentation for the exact dependency versions in the lockfile. Validate all untrusted inputs and keep secrets server-only. Add/adjust focused tests for this stage. Run relevant tests, type-check, lint and build where feasible. Never claim a check passed unless you ran it and report its actual result.

At completion, report: (1) changed files, (2) implementation decisions and assumptions, (3) commands run and pass/fail results, (4) acceptance criteria passed/failed/not tested, (5) risks or owner decisions, (6) next stage only if this gate passes. If blocked, report the minimal blocker and stop without guessing.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 03 - Username/password authentication and protected access

Secure the app before adding private drafts and uploads.

Objective

Secure the app before adding private drafts and uploads.

Preconditions

Read this TDD's applicable product, architecture and template sections before editing.

Confirm previous stages that this stage depends on are accepted; otherwise report the blocker and stop.

Inspect the live repository and fit implementation to actual code rather than assuming the example paths are exact.

Work items - do only these

Use a maintained authentication library compatible with the pinned Next.js version; do not hand-roll password storage, session tokens or cryptography.

Evaluate username-only sign-in against the library's actual registration/provisioning requirements. If internal email metadata is needed, document it and never expose it as a required login field unless approved.

Disable public self-registration for the initial app. Define an owner-controlled way to provision the first administrator and operator accounts.

Hash passwords using the library's secure default and configure safe session cookies, expiry, rotation/revocation and CSRF protections appropriate to its model.

Protect all app routes and server actions/API handlers. Enforce role/ownership checks on the server, not only in the UI.

Add login, logout, invalid-password, disabled-account and unauthenticated redirect behavior.

Use generic login errors that do not disclose whether a username exists; rate-limit repeated attempts where supported.

Files and modules in scope

Auth configuration and server-only auth utilities.

Login route/page and logout action.

Protected-route/server guard and minimal user-role contract.

Document a secure, one-time bootstrap method for first admin account.

BOUNDARY

Implement only authentication and access guards. Do not implement draft CRUD or editor controls. Do not add public registration. Report how the first administrator will be provisioned, then stop.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 03 - Acceptance and Antigravity prompt

The stage is complete only when its evidence is reported and its gate is passed.

Acceptance criteria

Anonymous requests cannot read drafts, upload media or invoke poster services.
Correct credentials create a session; wrong credentials do not.
Session survives normal navigation and can be invalidated on logout.
The public UI has no create-account path.
Secrets and password hashes never appear in client payloads or logs.

Verification plan

Test valid/invalid login, logout, expired session and unauthenticated access.
Attempt direct server/API calls without a session and confirm rejection.
Verify role checks on server-side operations.
Check the selected auth library documentation for this exact version.

STOP CONDITION

Implement only authentication and access guards. Do not implement draft CRUD or editor controls. Do not add public registration. Report how the first administrator will be provisioned, then stop. Do not proceed to the next stage until the owner accepts this report.

Copy-ready Antigravity prompt

COPY-READY / TECHNICAL BLOCK

You are implementing Ozone Lounge Poster Generator TDD Stage 03: Username/password authentication and protected access. Implement only this stage. First inspect the current repository and the applicable TDD sections. Preserve working code and completed stage behavior. Stay within the listed scope; do not begin later stages or unrelated refactors.

Use current official documentation for the exact dependency versions in the lockfile. Validate all untrusted inputs and keep secrets server-only. Add/adjust focused tests for this stage. Run relevant tests, type-check, lint and build where feasible. Never claim a check passed unless you ran it and report its actual result.

At completion, report: (1) changed files, (2) implementation decisions and assumptions, (3) commands run and pass/fail results, (4) acceptance criteria passed/failed/not tested, (5) risks or owner decisions, (6) next stage only if this gate passes. If blocked, report the minimal blocker and stop without guessing.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 04 - Responsive app shell and interface themes

Build the navigable mobile-first product frame without coupling app chrome to poster styling.

Objective

Build the navigable mobile-first product frame without coupling app chrome to poster styling.

Preconditions

- Read this TDD's applicable product, architecture and template sections before editing.
- Confirm previous stages that this stage depends on are accepted; otherwise report the blocker and stop.
- Inspect the live repository and fit implementation to actual code rather than assuming the example paths are exact.

Work items - do only these

- Create a layout optimized for narrow screens first, with usable touch targets, safe-area spacing and restrained desktop adaptations.
- Implement dark mode as the default app theme and a bright/light theme option.
- Persist the app-theme preference in an appropriate local preference mechanism; avoid placing it in poster document state.
- Add basic header with brand placeholder, current page title, theme toggle and profile/logout entry.
- Add accessible loading, empty, error and not-authorized states.
- Establish consistent buttons, text inputs, select controls, collapsible sections, dialogs and toast/status patterns.
- Keep gallery/editor routes protected through the auth boundary.

Files and modules in scope

- App layout/shell, theme tokens, reusable UI primitives and navigation.
- Theme preference handling and responsive breakpoints.
- Login page styling aligned with the shell, without expanding auth logic.

BOUNDARY

Implement only the responsive shell, theme system and reusable UI primitives. Use placeholder content where needed. Do not build the template gallery or editing engine yet.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 04 - Acceptance and Antigravity prompt

The stage is complete only when its evidence is reported and its gate is passed.

Acceptance criteria

Default app appearance is dark; user can switch to bright theme and preference is remembered.
Theme changes do not alter the eventual poster canvas, palette or export output.
No horizontal overflow at common phone widths.
Focus state, labels and basic keyboard navigation work.

Verification plan

Inspect at 360px, 390px, tablet and desktop widths.
Test theme toggle, reload persistence, focus visibility and navigation.
Run accessibility checks available in the project and production build.

STOP CONDITION

Implement only the responsive shell, theme system and reusable UI primitives. Use placeholder content where needed. Do not build the template gallery or editing engine yet. Do not proceed to the next stage until the owner accepts this report.

Copy-ready Antigravity prompt

COPY-READY / TECHNICAL BLOCK

You are implementing Ozone Lounge Poster Generator TDD Stage 04: Responsive app shell and interface themes. Implement only this stage. First inspect the current repository and the applicable TDD sections. Preserve working code and completed stage behavior. Stay within the listed scope; do not begin later stages or unrelated refactors.

Use current official documentation for the exact dependency versions in the lockfile. Validate all untrusted inputs and keep secrets server-only. Add/adjust focused tests for this stage. Run relevant tests, type-check, lint and build where feasible. Never claim a check passed unless you ran it and report its actual result.

At completion, report: (1) changed files, (2) implementation decisions and assumptions, (3) commands run and pass/fail results, (4) acceptance criteria passed/failed/not tested, (5) risks or owner decisions, (6) next stage only if this gate passes. If blocked, report the minimal blocker and stop without guessing.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 05 - Typed poster document contract and template registry

Create the extensible single source of truth for seven templates and their editable fields.

Objective

Create the extensible single source of truth for seven templates and their editable fields.

Preconditions

Read this TDD's applicable product, architecture and template sections before editing.

Confirm previous stages that this stage depends on are accepted; otherwise report the blocker and stop.

Inspect the live repository and fit implementation to actual code rather than assuming the example paths are exact.

Work items - do only these

Define a serialisable TypeScript poster document with schema version, template ID, event content, assets, style options, output metadata and revision metadata.

Validate untrusted stored/input state at runtime with a schema library such as Zod if compatible with the existing stack.

Create one registry containing id, day, template name, catalogue tag, event title, default performers, source-backed offers, creative description, allowed palettes/fonts, image slots and default document factory.

Keep template IDs stable and distinct from display names. Display names are Reddington, Saint, Lannister, Shelby, Lothbrok, Escobar and Pinkman.

Centralise template ordering in Monday-to-Sunday order; gallery cards, draft labels, export naming and editor selection must consume the registry.

Set fields for unknown logistics to empty optional values. Preserve Friday's empty offer as valid.

Add content regression tests matching the supplied weekly poster outline exactly.

Files and modules in scope

src/features/poster/schema.ts and document types.

src/features/templates/registry.ts plus seven metadata/default definitions.

Validation utilities and source-content fixtures/tests.

Document ID/version strategy for future migration.

BOUNDARY

Implement only types, validation and registry/default data. Do not build visual renderers or editor controls. Preserve exact source wording and report any ambiguity instead of inventing text.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 05 - Acceptance and Antigravity prompt

The stage is complete only when its evidence is reported and its gate is passed.

Acceptance criteria

All seven entries exist exactly once and appear in weekday order.
Registry can support an eighth template without editing multiple hard-coded switch statements.
Unknown details remain blank and conditional.
Runtime validation rejects invalid template IDs and malformed content.

Verification plan

Run type-check and registry/schema unit tests.
Compare every day, event, lineup and offer with the supplied outline.
Test a new unknown template ID and wrong field types.
Verify app theme and preview zoom are absent from saved poster state.

STOP CONDITION

Implement only types, validation and registry/default data. Do not build visual renderers or editor controls. Preserve exact source wording and report any ambiguity instead of inventing text. Do not proceed to the next stage until the owner accepts this report.

Copy-ready Antigravity prompt

COPY-READY / TECHNICAL BLOCK

You are implementing Ozone Lounge Poster Generator TDD Stage 05: Typed poster document contract and template registry. Implement only this stage. First inspect the current repository and the applicable TDD sections. Preserve working code and completed stage behavior. Stay within the listed scope; do not begin later stages or unrelated refactors.

Use current official documentation for the exact dependency versions in the lockfile. Validate all untrusted inputs and keep secrets server-only. Add/adjust focused tests for this stage. Run relevant tests, type-check, lint and build where feasible. Never claim a check passed unless you ran it and report its actual result.

At completion, report: (1) changed files, (2) implementation decisions and assumptions, (3) commands run and pass/fail results, (4) acceptance criteria passed/failed/not tested, (5) risks or owner decisions, (6) next stage only if this gate passes. If blocked, report the minimal blocker and stop without guessing.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 06 - Premium fonts and image asset infrastructure

Load licensed fonts and user-selected images predictably in editor, saved drafts and exported output.

Objective

Load licensed fonts and user-selected images predictably in editor, saved drafts and exported output.

Preconditions

Read this TDD's applicable product, architecture and template sections before editing.

Confirm previous stages that this stage depends on are accepted; otherwise report the blocker and stop.

Inspect the live repository and fit implementation to actual code rather than assuming the example paths are exact.

Work items - do only these

Create a font inventory that records display name, family ID, file path, file format, available weights/styles, licence/usage notes and status.

Support TTF and OTF files placed in the agreed public or asset location; use stable font-family IDs independent of filenames.

Avoid loading every font immediately. Load/register the selected font and required weights without causing late layout shifts.

Define image slots per template, including validation, accepted MIME types, size limits and dimension warnings.

Implement local object-URL previews with explicit cleanup/revocation, and distinguish ephemeral local preview URLs from durable stored URLs.

Provide storage adapter interfaces and select the approved provider through configuration; protect write credentials on the server.

Plan image crop/position metadata as document data; do not persist File objects or browser blobs in JSON.

Files and modules in scope

Font catalogue and registration utilities, fonts/ inventory and asset policy docs.

Client upload/preprocessing component and server upload/storage adapter.

Media metadata model and storage URL validation.

Placeholders or sample fonts only for development; production fonts supplied/licensed by the owner.

BOUNDARY

Implement only the font and image infrastructure. Do not design the seven templates yet. Do not include private font files from outside the approved repository or claim unprovided licences are cleared.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 06 - Acceptance and Antigravity prompt

The stage is complete only when its evidence is reported and its gate is passed.

Acceptance criteria

TTF/OTF fonts render with the expected selected weights in the live canvas.

Images preview immediately, reject unsafe/oversized files with understandable errors and survive save through durable URLs.

Temporary object URLs are cleaned up and never assumed valid after reload.

Font or asset failure falls back gracefully and is visible to the operator.

Verification plan

Test at least one TTF and one OTF file.

Upload, crop, save, reload and render a sample image.

Test unsupported MIME type, large file, missing asset and blocked URL.

Confirm cross-origin sources are not relied on without CORS-safe handling.

STOP CONDITION

Implement only the font and image infrastructure. Do not design the seven templates yet. Do not include private font files from outside the approved repository or claim unprovided licences are cleared. Do not proceed to the next stage until the owner accepts this report.

Copy-ready Antigravity prompt

COPY-READY / TECHNICAL BLOCK

You are implementing Ozone Lounge Poster Generator TDD Stage 06: Premium fonts and image asset infrastructure. Implement only this stage. First inspect the current repository and the applicable TDD sections. Preserve working code and completed stage behavior. Stay within the listed scope; do not begin later stages or unrelated refactors.

Use current official documentation for the exact dependency versions in the lockfile. Validate all untrusted inputs and keep secrets server-only. Add/adjust focused tests for this stage. Run relevant tests, type-check, lint and build where feasible. Never claim a check passed unless you ran it and report its actual result.

At completion, report: (1) changed files, (2) implementation decisions and assumptions, (3) commands run and pass/fail results, (4) acceptance criteria passed/failed/not tested, (5) risks or owner decisions, (6) next stage only if this gate passes. If blocked, report the minimal blocker and stop without guessing.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 07 - Fixed poster canvas, responsive scaling and export proof of concept

Prove that the poster can render at master dimensions while fitting a mobile viewport.

Objective

Prove that the poster can render at master dimensions while fitting a mobile viewport.

Preconditions

Read this TDD's applicable product, architecture and template sections before editing.

Confirm previous stages that this stage depends on are accepted; otherwise report the blocker and stop.

Inspect the live repository and fit implementation to actual code rather than assuming the example paths are exact.

Work items - do only these

Build a standalone fixed-coordinate canvas component with configured masterWidth/masterHeight values; use 1080 x 1350 only as a provisional value until the publishing channel is approved.

Use an outer wrapper whose actual layout size equals the scaled visual size and an inner fixed-size canvas transformed from the origin.

Calculate scale using the minimum available width/height ratio, with padding and ResizeObserver or equivalent resize awareness.

Keep toolbar, hints and zoom controls outside the scaled artwork DOM.

Create a minimal export proof with a test background, text, logo placeholder and selected custom font.

Use a rendering adapter so html-to-image or an alternative can be swapped without changing editor state.

Record browser, font-loading, CORS and large-image limitations discovered by the proof.

Files and modules in scope

PosterCanvas/CanvasViewport components, dimension constants/configuration and scale hook.

Exporter adapter interface and a disposable diagnostic export route/component.

Unit tests for scaling calculation.

BOUNDARY

Implement only the fixed canvas, viewport scaling and export proof. Do not build the full template editor or seven unique layouts. If browser rasterisation is unreliable, document the evidence and propose an isolated replacement strategy.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 07 - Acceptance and Antigravity prompt

The stage is complete only when its evidence is reported and its gate is passed.

Acceptance criteria

Canvas scales correctly on narrow, tall and desktop containers without pushing layout out of view.
Exported test file has exact configured master dimensions regardless of screen size or current scale.
Export excludes controls and app background.
A failure is reported clearly; it does not create a blank file silently.

Verification plan

Test scale math with known container dimensions.
Export PNG and JPEG test fixtures and inspect dimensions via image metadata.
Compare live canvas to export for font, image, clipping and positioning.
Test on at least one real target mobile browser before marking renderer viable.

STOP CONDITION

Implement only the fixed canvas, viewport scaling and export proof. Do not build the full template editor or seven unique layouts. If browser rasterisation is unreliable, document the evidence and propose an isolated replacement strategy. Do not proceed to the next stage until the owner accepts this report.

Copy-ready Antigravity prompt

COPY-READY / TECHNICAL BLOCK

You are implementing Ozone Lounge Poster Generator TDD Stage 07: Fixed poster canvas, responsive scaling and export proof of concept. Implement only this stage. First inspect the current repository and the applicable TDD sections. Preserve working code and completed stage behavior. Stay within the listed scope; do not begin later stages or unrelated refactors.

Use current official documentation for the exact dependency versions in the lockfile. Validate all untrusted inputs and keep secrets server-only. Add/adjust focused tests for this stage. Run relevant tests, type-check, lint and build where feasible. Never claim a check passed unless you ran it and report its actual result.

At completion, report: (1) changed files, (2) implementation decisions and assumptions, (3) commands run and pass/fail results, (4) acceptance criteria passed/failed/not tested, (5) risks or owner decisions, (6) next stage only if this gate passes. If blocked, report the minimal blocker and stop without guessing.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 08 - Shared editor state and mobile control framework

Build reusable editing controls and a clean state-update path without designing template-specific artwork.

Objective

Build reusable editing controls and a clean state-update path without designing template-specific artwork.

Preconditions

Read this TDD's applicable product, architecture and template sections before editing.

Confirm previous stages that this stage depends on are accepted; otherwise report the blocker and stop.

Inspect the live repository and fit implementation to actual code rather than assuming the example paths are exact.

Work items - do only these

Define a single poster document state with typed actions for content, palette, typography, assets, crop settings and optional elements.

Separate transient UI state such as open accordion, active field, theme, preview zoom and unsaved dialog from saved poster design state.

Create mobile-first editor sections for Event Details, Lineup, Offer, Artwork, Typography, Colours and Publishing Details.

Render template-specific controls from each template’s schema/allowlist instead of exposing every possible option to every template.

Add dirty-state tracking and navigation warnings for unsaved edits.

Apply input validation and sensible text-length guidance; avoid silent truncation of lineup or offer copy.

Ensure controls are usable with keyboard and touch and have explicit labels/help text.

Files and modules in scope

Poster editor state/context or reducer, field controls and template-configuration section components.

Dirty state and form validation utilities.

Unit tests for state reducer/actions and document transformations.

BOUNDARY

Implement only editor state and reusable controls. Do not create poster designs or database save actions. Keep the document serialisable and add tests before connecting real templates.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 08 - Acceptance and Antigravity prompt

The stage is complete only when its evidence is reported and its gate is passed.

Acceptance criteria

Editing controls update live document state predictably.
Transient app theme and preview zoom never dirty or modify saved poster state.
Changing template-specific options cannot create invalid document objects.
Long text provides a warning/reflow strategy rather than silently disappearing.

Verification plan

Test reducer updates and runtime validation.
Exercise each control with mouse and touch emulator.
Reload a saved fixture and confirm option controls map to the document correctly.
Check dirty-state prompt for navigation and discard/save paths.

STOP CONDITION

Implement only editor state and reusable controls. Do not create poster designs or database save actions. Keep the document serialisable and add tests before connecting real templates. Do not proceed to the next stage until the owner accepts this report.

Copy-ready Antigravity prompt

COPY-READY / TECHNICAL BLOCK

You are implementing Ozone Lounge Poster Generator TDD Stage 08: Shared editor state and mobile control framework. Implement only this stage. First inspect the current repository and the applicable TDD sections. Preserve working code and completed stage behavior. Stay within the listed scope; do not begin later stages or unrelated refactors.

Use current official documentation for the exact dependency versions in the lockfile. Validate all untrusted inputs and keep secrets server-only. Add/adjust focused tests for this stage. Run relevant tests, type-check, lint and build where feasible. Never claim a check passed unless you ran it and report its actual result.

At completion, report: (1) changed files, (2) implementation decisions and assumptions, (3) commands run and pass/fail results, (4) acceptance criteria passed/failed/not tested, (5) risks or owner decisions, (6) next stage only if this gate passes. If blocked, report the minimal blocker and stop without guessing.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 09 - Reddington end-to-end vertical slice

Complete one genuinely polished template through login, editing, assets, save path stub or service, preview and export integration.

Objective

Complete one genuinely polished template through login, editing, assets, save path stub or service, preview and export integration.

Preconditions

Read this TDD's applicable product, architecture and template sections before editing.
Confirm previous stages that this stage depends on are accepted; otherwise report the blocker and stop.
Inspect the live repository and fit implementation to actual code rather than assuming the example paths are exact.

Work items - do only these

Build the Monday Reddington renderer as a distinct retro/sunset/vintage-music composition.
Expose Monday event copy and lineup exactly as supplied, with editable date/time/logistics fields left blank initially.
Implement approved palette choices, supported display/body font options and meaningful layout options, not unrestricted free positioning.
Use a separate logo layer with aspect-ratio preservation and a readable safe zone.
Add template thumbnail/card metadata and sensible default image placeholder that cannot be mistaken for approved production photography.
Wire editor controls to live preview and export POC; keep data service integration aligned to stage 11 if not yet ready.
Create visual fixtures for default, long title, long lineup, missing image and future offer-free layout.

Files and modules in scope

ReddingtonRenderer and Monday editor schema/controls.
Reddington palette and typography presets and template preview thumbnail.
Visual/test fixtures and content assertions.

BOUNDARY

Implement only Reddington as the first template vertical slice. Reuse the approved canvas/registry/editor contracts. Do not build the remaining six designs or refactor unrelated shared components.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 09 - Acceptance and Antigravity prompt

The stage is complete only when its evidence is reported and its gate is passed.

Acceptance criteria

Reddington reads as a distinct Monday retro identity, not a generic recoloured poster.
Source event/lineup/offer wording is correct; wine exception remains explicit.
Logo, text, crop and offer are legible at phone preview and master-size export.
Changing a curated option changes both preview and export.

Verification plan

Render default, long-copy and missing-image fixtures.
Verify no text runs beyond canvas bounds or overlaps critical content.
Export and inspect master dimensions, fonts, image and shadow treatment.
Compare with owner-approved reference once assets are supplied.

STOP CONDITION

Implement only Reddington as the first template vertical slice. Reuse the approved canvas/registry/editor contracts. Do not build the remaining six designs or refactor unrelated shared components. Do not proceed to the next stage until the owner accepts this report.

Copy-ready Antigravity prompt

COPY-READY / TECHNICAL BLOCK

You are implementing Ozone Lounge Poster Generator TDD Stage 09: Reddington end-to-end vertical slice. Implement only this stage. First inspect the current repository and the applicable TDD sections. Preserve working code and completed stage behavior. Stay within the listed scope; do not begin later stages or unrelated refactors.

Use current official documentation for the exact dependency versions in the lockfile. Validate all untrusted inputs and keep secrets server-only. Add/adjust focused tests for this stage. Run relevant tests, type-check, lint and build where feasible. Never claim a check passed unless you ran it and report its actual result.

At completion, report: (1) changed files, (2) implementation decisions and assumptions, (3) commands run and pass/fail results, (4) acceptance criteria passed/failed/not tested, (5) risks or owner decisions, (6) next stage only if this gate passes. If blocked, report the minimal blocker and stop without guessing.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 10 - Full-screen poster preview and touch zoom

Provide the requested immersive preview without changing design or export geometry.

Objective

Provide the requested immersive preview without changing design or export geometry.

Preconditions

Read this TDD's applicable product, architecture and template sections before editing.

Confirm previous stages that this stage depends on are accepted; otherwise report the blocker and stop.

Inspect the live repository and fit implementation to actual code rather than assuming the example paths are exact.

Work items - do only these

Add an explicit Preview action in the editor header, reachable on mobile.

Open a full-screen overlay or route that uses the exact same poster document/renderer, not a screenshot of the scaled editor.

Implement pinch-to-zoom for touch, pan when zoomed, plus/minus buttons, Fit-to-screen and reset-to-fit behavior.

Place a clear X close control in a safe-area-aware header; closing returns to the same editor state and scroll/selection where feasible.

Keep preview controls outside artwork and prevent gestures from accidentally editing fields behind the overlay.

Use accessible labels and keyboard escape/close behavior.

Make zoom/pan transient view state only; no effect on saved canvas dimensions or export output.

Files and modules in scope

FullScreenPreview component/route, touch gesture and zoom state utility.

Responsive safe-area layout and a11y labels.

Touch/keyboard tests.

BOUNDARY

Implement only full-screen preview and navigation/gesture behavior. Do not add unrelated editor features. Explicitly test that zoom/pan never mutates the design document or the exporter’s master dimensions.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 10 - Acceptance and Antigravity prompt

The stage is complete only when its evidence is reported and its gate is passed.

Acceptance criteria

Preview fills the device view without hidden overflow and poster remains fully accessible via Fit.
Pinch zoom and pan work on touch; plus/minus and Fit work with mouse/keyboard.
X/Escape returns to editor without losing current unsaved changes.
Export is identical before and after preview zoom operations.

Verification plan

Test preview open/close, pinch, pan, zoom buttons and Fit.
Test small viewport and device safe-area constraints.
Compare exported image hash/pixels before and after arbitrary zoom/pan operations.
Verify modal focus and close behavior.

STOP CONDITION

Implement only full-screen preview and navigation/gesture behavior. Do not add unrelated editor features. Explicitly test that zoom/pan never mutates the design document or the exporter's master dimensions. Do not proceed to the next stage until the owner accepts this report.

Copy-ready Antigravity prompt

COPY-READY / TECHNICAL BLOCK

You are implementing Ozone Lounge Poster Generator TDD Stage 10: Full-screen poster preview and touch zoom. Implement only this stage. First inspect the current repository and the applicable TDD sections. Preserve working code and completed stage behavior. Stay within the listed scope; do not begin later stages or unrelated refactors.

Use current official documentation for the exact dependency versions in the lockfile. Validate all untrusted inputs and keep secrets server-only. Add/adjust focused tests for this stage. Run relevant tests, type-check, lint and build where feasible. Never claim a check passed unless you ran it and report its actual result.

At completion, report: (1) changed files, (2) implementation decisions and assumptions, (3) commands run and pass/fail results, (4) acceptance criteria passed/failed/not tested, (5) risks or owner decisions, (6) next stage only if this gate passes. If blocked, report the minimal blocker and stop without guessing.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 11 - Draft persistence and lifecycle

Make drafts durable so work survives refresh, relogin and device changes.

Objective

Make drafts durable so work survives refresh, relogin and device changes.

Preconditions

Read this TDD's applicable product, architecture and template sections before editing.
Confirm previous stages that this stage depends on are accepted; otherwise report the blocker and stop.
Inspect the live repository and fit implementation to actual code rather than assuming the example paths are exact.

Work items - do only these

Implement create, list, open, update, duplicate and delete draft operations using server-only validated services.
Validate and normalise every incoming document, template ID and asset reference before storing it.
Define and enforce the owner/shared-workspace policy; until approved, default to private owner-scoped drafts.
Save stable HTTPS storage references, not browser object URLs or raw File objects.
Use stable draft IDs and updated timestamps; ensure Save updates the current ID rather than creating duplicate drafts.
Add save status, error recovery, explicit retry, unsaved-change warnings and safe deletion confirmation.
Use optimistic UI only where failure can be rolled back; never report success before persistence confirms.

Files and modules in scope

Draft service/repository and server actions/API routes.
Draft list, save/open and duplicate/delete UI.
Schema migration if needed and ownership tests.
Document URL query approach if it fits the app router; preserve native navigation conventions.

BOUNDARY

Implement only draft persistence/lifecycle. If draft sharing policy remains undecided, implement private ownership first and document the migration implications before any shared-team access.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 11 - Acceptance and Antigravity prompt

The stage is complete only when its evidence is reported and its gate is passed.

Acceptance criteria

A saved draft reopens correctly after refresh and a new login session.
No user can read/update/delete another user's private drafts.
Images/fonts/options and schema version restore correctly.
Save failures leave recoverable local state and show an actionable status.
Delete is protected against accidental taps and cross-owner IDs.

Verification plan

Test create/update/reopen/duplicate/delete and invalid documents.
Test two separate users and attempt ID substitution.
Test asset URL expiry/unavailability and malformed stored JSON.
Verify timestamps and update-in-place behavior.

STOP CONDITION

Implement only draft persistence/lifecycle. If draft sharing policy remains undecided, implement private ownership first and document the migration implications before any shared-team access. Do not proceed to the next stage until the owner accepts this report.

Copy-ready Antigravity prompt

COPY-READY / TECHNICAL BLOCK

You are implementing Ozone Lounge Poster Generator TDD Stage 11: Draft persistence and lifecycle. Implement only this stage. First inspect the current repository and the applicable TDD sections. Preserve working code and completed stage behavior. Stay within the listed scope; do not begin later stages or unrelated refactors.

Use current official documentation for the exact dependency versions in the lockfile. Validate all untrusted inputs and keep secrets server-only. Add/adjust focused tests for this stage. Run relevant tests, type-check, lint and build where feasible. Never claim a check passed unless you ran it and report its actual result.

At completion, report: (1) changed files, (2) implementation decisions and assumptions, (3) commands run and pass/fail results, (4) acceptance criteria passed/failed/not tested, (5) risks or owner decisions, (6) next stage only if this gate passes. If blocked, report the minimal blocker and stop without guessing.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 12 - Production export pipeline and mobile verification

Harden export for reliable PNG/JPEG generation from the master canvas.

Objective

Harden export for reliable PNG/JPEG generation from the master canvas.

Preconditions

- Read this TDD's applicable product, architecture and template sections before editing.
- Confirm previous stages that this stage depends on are accepted; otherwise report the blocker and stop.
- Inspect the live repository and fit implementation to actual code rather than assuming the example paths are exact.

Work items - do only these

- Expose PNG and high-quality JPEG export actions with file names derived from date/day/template title; avoid unsafe characters.
- Await selected font readiness and asset decoding before rasterising.
- Export the unscaled master artwork at its exact configured width and height; do not export a viewport screenshot.
- Ensure cross-origin images, CSS filters, font embedding and SVG elements behave in the selected rasteriser; avoid tainted canvases.
- Show progress, disable duplicate clicks, capture known errors and allow retry without losing edits.
- Use sensible JPEG quality and PNG alpha handling; verify MIME/extension match actual bytes.
- Document mobile download/share limitations and implement native Web Share when supported only as a progressive enhancement, with download fallback.

Files and modules in scope

- Final export service/adapter and export menu.
- Filename sanitation and publishing preflight helper.
- Export diagnostics and fixture suite.

BOUNDARY

Harden only the export path and run mobile verification. Do not build further template renderers until export is reliable. If html-to-image fails in a target browser, report reproducible evidence and compare an isolated alternative rather than adding random workarounds.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 12 - Acceptance and Antigravity prompt

The stage is complete only when its evidence is reported and its gate is passed.

Acceptance criteria

PNG/JPEG dimensions match configured master dimensions exactly.
File opens in standard image viewers and matches extension/MIME.
Font, images, logo, crop and palette match the live master canvas.
Export excludes controls, preview chrome and UI theme.
Failures never trigger a success toast or corrupt the draft.

Verification plan

Test selected fonts before export and forced font/image load failures.
Test PNG/JPEG on desktop and a real iOS or Android target if available.
Check image dimensions and manually inspect representative exports.
Test large image/file-size behavior and repeated export actions.

STOP CONDITION

Harden only the export path and run mobile verification. Do not build further template renderers until export is reliable. If html-to-image fails in a target browser, report reproducible evidence and compare an isolated alternative rather than adding random workarounds. Do not proceed to the next stage until the owner accepts this report.

Copy-ready Antigravity prompt

COPY-READY / TECHNICAL BLOCK

You are implementing Ozone Lounge Poster Generator TDD Stage 12: Production export pipeline and mobile verification. Implement only this stage. First inspect the current repository and the applicable TDD sections. Preserve working code and completed stage behavior. Stay within the listed scope; do not begin later stages or unrelated refactors.

Use current official documentation for the exact dependency versions in the lockfile. Validate all untrusted inputs and keep secrets server-only. Add/adjust focused tests for this stage. Run relevant tests, type-check, lint and build where feasible. Never claim a check passed unless you ran it and report its actual result.

At completion, report: (1) changed files, (2) implementation decisions and assumptions, (3) commands run and pass/fail results, (4) acceptance criteria passed/failed/not tested, (5) risks or owner decisions, (6) next stage only if this gate passes. If blocked, report the minimal blocker and stop without guessing.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 13 - Saint - Tuesday African Night renderer

Build a genuinely separate Afrobeat/African-inspired Tuesday template.

Objective

Build a genuinely separate Afrobeat/African-inspired Tuesday template.

Preconditions

Read this TDD's applicable product, architecture and template sections before editing.
Confirm previous stages that this stage depends on are accepted; otherwise report the blocker and stop.
Inspect the live repository and fit implementation to actual code rather than assuming the example paths are exact.

Work items - do only these

Create Saint’s own composition using bold graphic geometry, rhythm, pattern accents and confident title hierarchy.
Use source text: African Night (Afro); DJs DJ Neb & DVJ Jojez; discounts on beer buckets and cocktails.
Offer several coordinated colour palettes and suitable display/body font pairings available in the curated font catalogue.
Support an approved hero photo, crop/position and optional controlled pattern intensity.
Keep decorative motifs behind content and ensure contrast remains high across images and palettes.
Use registry metadata and shared state/export contracts; avoid copying Reddington markup and only swapping colours.

Files and modules in scope

Saint renderer, schema/option config, thumbnail and palettes.
Template-specific visual fixtures and content tests.

BOUNDARY

Implement only Saint’s unique renderer and template options. Do not alter shared Reddington behavior unless a reusable defect is identified and covered by regression tests.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 13 - Acceptance and Antigravity prompt

The stage is complete only when its evidence is reported and its gate is passed.

Acceptance criteria

Saint has a distinct modern Afrobeat identity, clearly different from vintage Reddington and neon Lannister.
DJ names, event name and offer are accurate and legible.
All options survive save/reopen and export.
Missing images and long text still produce usable output.

Verification plan

Compare template contact sheet against Reddington.
Test every palette with default and bright-background images.
Export master-size fixtures and inspect layout/contrast.
Run registry/document regression tests.

STOP CONDITION

Implement only Saint's unique renderer and template options. Do not alter shared Reddington behavior unless a reusable defect is identified and covered by regression tests. Do not proceed to the next stage until the owner accepts this report.

Copy-ready Antigravity prompt

COPY-READY / TECHNICAL BLOCK

You are implementing Ozone Lounge Poster Generator TDD Stage 13: Saint - Tuesday African Night renderer. Implement only this stage. First inspect the current repository and the applicable TDD sections. Preserve working code and completed stage behavior. Stay within the listed scope; do not begin later stages or unrelated refactors.

Use current official documentation for the exact dependency versions in the lockfile. Validate all untrusted inputs and keep secrets server-only. Add/adjust focused tests for this stage. Run relevant tests, type-check, lint and build where feasible. Never claim a check passed unless you ran it and report its actual result.

At completion, report: (1) changed files, (2) implementation decisions and assumptions, (3) commands run and pass/fail results, (4) acceptance criteria passed/failed/not tested, (5) risks or owner decisions, (6) next stage only if this gate passes. If blocked, report the minimal blocker and stop without guessing.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 14 - Lannister - Wednesday Groovy Wednesday renderer

Build the midweek neon/disco design and represent its multi-part promotion clearly.

Objective

Build the midweek neon/disco design and represent its multi-part promotion clearly.

Preconditions

Read this TDD's applicable product, architecture and template sections before editing.
Confirm previous stages that this stage depends on are accepted; otherwise report the blocker and stop.
Inspect the live repository and fit implementation to actual code rather than assuming the example paths are exact.

Work items - do only these

Create a composition around disco, neon accents, nightlife photography and dance-floor energy.
Use source content: Groovy Wednesday; DJ Donben & DJ Vin; free koroga chicken on the house; discounts on whiskey bottles, cognac and beer buckets.
Separate the complimentary food highlight from the bottle/bucket discount grouping so the offer is scannable.
Offer coordinated neon palette presets and curated type choices while preserving accessibility/legibility.
Support optional glow/light texture controls with performance-conscious defaults.
Avoid reusing Saint's geometric language as the primary composition.

Files and modules in scope

Lannister renderer, Wednesday field groups, palette/typography presets and thumbnail.
Source-copy assertions and visual fixtures.

BOUNDARY

Implement only Lannister. Do not invent a price, time, alcohol brand or other promotion. Keep the two promotional groupings semantically clear.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 14 - Acceptance and Antigravity prompt

The stage is complete only when its evidence is reported and its gate is passed.

Acceptance criteria

Free koroga chicken is prominent without changing the source wording.
All listed discount categories are present, none are fabricated.
Visual identity is distinct from both Reddington and Saint.
Glow effects do not disappear or become clipped in export.

Verification plan

Test content order, narrow title wrapping and high/low brightness photos.
Export with optional effects on/off and inspect fidelity.
Run content/schema tests and mobile-scale preview review.

STOP CONDITION

Implement only Lannister. Do not invent a price, time, alcohol brand or other promotion. Keep the two promotional groupings semantically clear. Do not proceed to the next stage until the owner accepts this report.

Copy-ready Antigravity prompt

COPY-READY / TECHNICAL BLOCK

You are implementing Ozone Lounge Poster Generator TDD Stage 14: Lannister - Wednesday Groovy Wednesday renderer. Implement only this stage. First inspect the current repository and the applicable TDD sections. Preserve working code and completed stage behavior. Stay within the listed scope; do not begin later stages or unrelated refactors.

Use current official documentation for the exact dependency versions in the lockfile. Validate all untrusted inputs and keep secrets server-only. Add/adjust focused tests for this stage. Run relevant tests, type-check, lint and build where feasible. Never claim a check passed unless you ran it and report its actual result.

At completion, report: (1) changed files, (2) implementation decisions and assumptions, (3) commands run and pass/fail results, (4) acceptance criteria passed/failed/not tested, (5) risks or owner decisions, (6) next stage only if this gate passes. If blocked, report the minimal blocker and stop without guessing.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 15 - Shelby - Thursday Karaoke Night renderer

Build a stage/microphone visual identity with accurate host and DJ roles.

Objective

Build a stage/microphone visual identity with accurate host and DJ roles.

Preconditions

Read this TDD's applicable product, architecture and template sections before editing.
Confirm previous stages that this stage depends on are accepted; otherwise report the blocker and stop.
Inspect the live repository and fit implementation to actual code rather than assuming the example paths are exact.

Work items - do only these

Create a karaoke composition using microphone/stage imagery, spotlight and a lively sing-along atmosphere.
Display Hosts: Bellah Harriet & Tito Wagithomo and DJ: DJ Fab as distinct role groupings.
Use source promotion: discounts on gin, rum and vodka; free tequila shots for singers.
Offer palettes and typography pairs suitable for dramatic stage lighting.
Make role labels editable without collapsing all names into one ambiguous lineup string.
Ensure offer badges stay legible against spotlight gradients and photographic backgrounds.

Files and modules in scope

Shelby renderer, separate host and DJ fields, layout presets, thumbnail and tests.
Document migration/default transformation if a single-line lineup was used in prototypes.

BOUNDARY

Implement only Shelby. Preserve the different host and DJ roles in the data model and UI. Do not silently compress the lineup in a way that loses role meaning.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 15 - Acceptance and Antigravity prompt

The stage is complete only when its evidence is reported and its gate is passed.

Acceptance criteria

Host and DJ roles are visually and semantically distinguishable.
All promotions render accurately, with tequila shots clearly tied to singers.
Shelby uses a distinct karaoke/stage language, not only a colour swap.
Missing optional host photo/logo never breaks layout.

Verification plan

Test long host names, one host missing and all names present.
Test contrast with light/dark stage photos and export parity.
Verify content assertions against the original outline.

STOP CONDITION

Implement only Shelby. Preserve the different host and DJ roles in the data model and UI. Do not silently compress the lineup in a way that loses role meaning. Do not proceed to the next stage until the owner accepts this report.

Copy-ready Antigravity prompt

COPY-READY / TECHNICAL BLOCK

You are implementing Ozone Lounge Poster Generator TDD Stage 15: Shelby - Thursday Karaoke Night renderer. Implement only this stage. First inspect the current repository and the applicable TDD sections. Preserve working code and completed stage behavior. Stay within the listed scope; do not begin later stages or unrelated refactors.

Use current official documentation for the exact dependency versions in the lockfile. Validate all untrusted inputs and keep secrets server-only. Add/adjust focused tests for this stage. Run relevant tests, type-check, lint and build where feasible. Never claim a check passed unless you ran it and report its actual result.

At completion, report: (1) changed files, (2) implementation decisions and assumptions, (3) commands run and pass/fail results, (4) acceptance criteria passed/failed/not tested, (5) risks or owner decisions, (6) next stage only if this gate passes. If blocked, report the minimal blocker and stop without guessing.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 16 - Lothbrok - Friday Flight Fridays renderer

Create a premium Friday design that remains balanced when the promotion is blank.

Objective

Create a premium Friday design that remains balanced when the promotion is blank.

Preconditions

Read this TDD's applicable product, architecture and template sections before editing.
Confirm previous stages that this stage depends on are accepted; otherwise report the blocker and stop.
Inspect the live repository and fit implementation to actual code rather than assuming the example paths are exact.

Work items - do only these

Create a premium nightlife composition using strong DJ/hero photography, restrained metallic/light accents and energetic diagonal or split framing.
Use source content: Flight Fridays; DJ Slim D & Mistabrand.
Keep offer empty in the canonical default. Do not render “No special offer provided” or an empty offer card.
Rebalance layout when an owner-approved offer is added later, with a test variation to prove both states.
Offer only curated palette/typography/crop choices consistent with the premium mood.
Do not invent drink deals, entry pricing, dress code or additional performers.

Files and modules in scope

Lothbrok renderer, optional offer component behavior, Friday presets and fixtures.
Blank-offer and future-offer visual tests.

BOUNDARY

Implement only Lothbrok. Treat missing Friday offer as valid content, not an error. Do not fill the gap by inventing copy; make the offer area conditional.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 16 - Acceptance and Antigravity prompt

The stage is complete only when its evidence is reported and its gate is passed.

Acceptance criteria

Default has no visible empty offer placeholder.
Friday event and DJ names are correct.
The no-offer state remains visually intentional and well balanced.
Adding a future offer is possible without editing shared exporter code.

Verification plan

Compare blank-offer and populated-offer fixtures.
Check no source-text invention in rendered default.
Inspect small viewport preview and master export.
Run content regression and registry tests.

STOP CONDITION

Implement only Lothbrok. Treat missing Friday offer as valid content, not an error. Do not fill the gap by inventing copy; make the offer area conditional. Do not proceed to the next stage until the owner accepts this report.

Copy-ready Antigravity prompt

COPY-READY / TECHNICAL BLOCK

You are implementing Ozone Lounge Poster Generator TDD Stage 16: Lothbrok - Friday Flight Fridays renderer. Implement only this stage. First inspect the current repository and the applicable TDD sections. Preserve working code and completed stage behavior. Stay within the listed scope; do not begin later stages or unrelated refactors.

Use current official documentation for the exact dependency versions in the lockfile. Validate all untrusted inputs and keep secrets server-only. Add/adjust focused tests for this stage. Run relevant tests, type-check, lint and build where feasible. Never claim a check passed unless you ran it and report its actual result.

At completion, report: (1) changed files, (2) implementation decisions and assumptions, (3) commands run and pass/fail results, (4) acceptance criteria passed/failed/not tested, (5) risks or owner decisions, (6) next stage only if this gate passes. If blocked, report the minimal blocker and stop without guessing.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 17 - Escobar - Saturday Fiesta renderer

Create a colourful celebration-oriented weekend poster distinct from all prior layouts.

Objective

Create a colourful celebration-oriented weekend poster distinct from all prior layouts.

Preconditions

Read this TDD's applicable product, architecture and template sections before editing.
Confirm previous stages that this stage depends on are accepted; otherwise report the blocker and stop.
Inspect the live repository and fit implementation to actual code rather than assuming the example paths are exact.

Work items - do only these

Build layered colour, celebratory lights and vibrant party imagery using a different composition from Saint's African pattern system and Lannister's neon disco style.
Use source content: Saturday Fiesta; DVJ Jojez; discounts on whiskey bottles and cognac.
Add coordinated fiesta palette variants and optional texture/light intensity.
Keep headline and offer hierarchy clear over bright multi-colour backgrounds.
Use controlled decorative overlays; allow reduced-effect mode if user photo already contains strong light.
Maintain logo safe area and crop behavior across layouts.

Files and modules in scope

Escobar renderer, palettes, light/texture controls, thumbnail and tests.
Visual fixture set for vivid and restrained versions.

BOUNDARY

Implement only Escobar's renderer and controls. Avoid reusing the exact composition of other templates. Keep palette changes coordinated across relevant tokens.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 17 - Acceptance and Antigravity prompt

The stage is complete only when its evidence is reported and its gate is passed.

Acceptance criteria

Escobar reads as a celebratory Saturday identity rather than a colour-adjusted Saint or Lannister.
DVJ Jojez and both offer categories are accurate.
Brightness/contrast remain sufficient across palette options.
Layout remains complete with texture turned off.

Verification plan

Inspect every palette over vivid and dark image fixtures.
Export with effects on/off and check asset layers.
Run long-copy and no-image fallbacks.

STOP CONDITION

Implement only Escobar's renderer and controls. Avoid reusing the exact composition of other templates. Keep palette changes coordinated across relevant tokens. Do not proceed to the next stage until the owner accepts this report.

Copy-ready Antigravity prompt

COPY-READY / TECHNICAL BLOCK

You are implementing Ozone Lounge Poster Generator TDD Stage 17: Escobar - Saturday Fiesta renderer. Implement only this stage. First inspect the current repository and the applicable TDD sections. Preserve working code and completed stage behavior. Stay within the listed scope; do not begin later stages or unrelated refactors.

Use current official documentation for the exact dependency versions in the lockfile. Validate all untrusted inputs and keep secrets server-only. Add/adjust focused tests for this stage. Run relevant tests, type-check, lint and build where feasible. Never claim a check passed unless you ran it and report its actual result.

At completion, report: (1) changed files, (2) implementation decisions and assumptions, (3) commands run and pass/fail results, (4) acceptance criteria passed/failed/not tested, (5) risks or owner decisions, (6) next stage only if this gate passes. If blocked, report the minimal blocker and stop without guessing.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 18 - Pinkman - Sunday Old Skool renderer

Build a nostalgic, relaxed throwback design for Sunday.

Objective

Build a nostalgic, relaxed throwback design for Sunday.

Preconditions

Read this TDD's applicable product, architecture and template sections before editing.
Confirm previous stages that this stage depends on are accepted; otherwise report the blocker and stop.
Inspect the live repository and fit implementation to actual code rather than assuming the example paths are exact.

Work items - do only these

Create a relaxed old-school composition with vinyl/cassette/record-inspired motifs, textured print feel and warm/faded colour treatments.
Use source content: Old Skool Sundays; DJ Ngash; discounts on beer buckets.
Offer restrained nostalgic palettes such as amber, faded teal and brick/cream with legible headline/body pairings.
Keep decorative texture away from practical event copy and allow it to be reduced/disabled.
Avoid making the poster feel like Saturday Fiesta or Monday retro-sundowner despite shared vintage cues.
Ensure optional motif/photo layers gracefully disappear when assets are absent.

Files and modules in scope

Pinkman renderer, motif/texture options, palettes, thumbnail and tests.
Fixtures with textures enabled and disabled.

BOUNDARY

Implement only Pinkman. Keep the nostalgic mood distinct from Reddington by using Sunday's vinyl/throwback direction and its own layout grammar.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 18 - Acceptance and Antigravity prompt

The stage is complete only when its evidence is reported and its gate is passed.

Acceptance criteria

Sunday event, DJ and offer are accurate.
The design has a distinct, relaxed old-skool identity.
Text remains legible and motifs do not obstruct core information.
Save/reopen/export preserve motif settings and palette choices.

Verification plan

Compare contact sheet across all seven templates.
Test texture reduction, missing photo and long offer/content.
Inspect master-size exports for visible texture artefacts.

STOP CONDITION

Implement only Pinkman. Keep the nostalgic mood distinct from Reddington by using Sunday's vinyl/throwback direction and its own layout grammar. Do not proceed to the next stage until the owner accepts this report.

Copy-ready Antigravity prompt

COPY-READY / TECHNICAL BLOCK

You are implementing Ozone Lounge Poster Generator TDD Stage 18: Pinkman - Sunday Old Skool renderer. Implement only this stage. First inspect the current repository and the applicable TDD sections. Preserve working code and completed stage behavior. Stay within the listed scope; do not begin later stages or unrelated refactors.

Use current official documentation for the exact dependency versions in the lockfile. Validate all untrusted inputs and keep secrets server-only. Add/adjust focused tests for this stage. Run relevant tests, type-check, lint and build where feasible. Never claim a check passed unless you ran it and report its actual result.

At completion, report: (1) changed files, (2) implementation decisions and assumptions, (3) commands run and pass/fail results, (4) acceptance criteria passed/failed/not tested, (5) risks or owner decisions, (6) next stage only if this gate passes. If blocked, report the minimal blocker and stop without guessing.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 19 - Gallery, template switching and cross-template polish

Finish the discoverability and consistency layer after all seven renderers work.

Objective

Finish the discoverability and consistency layer after all seven renderers work.

Preconditions

- Read this TDD's applicable product, architecture and template sections before editing.
- Confirm previous stages that this stage depends on are accepted; otherwise report the blocker and stop.
- Inspect the live repository and fit implementation to actual code rather than assuming the example paths are exact.

Work items - do only these

- Build a Monday-to-Sunday gallery entirely from the single registry, including template name, event name, best-for tag and thumbnail.
- Add clear selected-template details and draft entry points without exposing nonexistent capabilities.
- Ensure switching a template creates or opens an appropriate document intentionally; never discard unsaved changes silently.
- Add draft filtering/search/sort only if supported by current scope and persisted data.
- Create a visual contact sheet and content matrix for all seven templates.
- Normalize button labels, error messages, preview affordances and field behaviors while preserving unique designs.
- Confirm all template versions/defaults and legacy draft fallbacks are deterministic.

Files and modules in scope

- Gallery/list pages and registry-driven card component.
- Template switching guard, shared empty/loading/error states and visual regression fixtures.
- Cross-template test suite.

BOUNDARY

Implement only the gallery and cross-template integration/polish. Do not restyle a template without a documented issue and a reference screenshot. Preserve the registry as the only catalogue source.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 19 - Acceptance and Antigravity prompt

The stage is complete only when its evidence is reported and its gate is passed.

Acceptance criteria

Seven templates appear in weekday order with correct tags and content.
Each template is visually distinct in a contact sheet.
Switching template never silently loses a dirty document.
Gallery does not require a new hard-coded entry when template nine is introduced.

Verification plan

Test all seven cards, selection, switch and back navigation.
Run the same core save/preview/export test for every template.
Review mobile responsive contact sheet and keyboard flow.
Run full type-check, lint, tests and production build.

STOP CONDITION

Implement only the gallery and cross-template integration/polish. Do not restyle a template without a documented issue and a reference screenshot. Preserve the registry as the only catalogue source. Do not proceed to the next stage until the owner accepts this report.

Copy-ready Antigravity prompt

COPY-READY / TECHNICAL BLOCK

You are implementing Ozone Lounge Poster Generator TDD Stage 19: Gallery, template switching and cross-template polish. Implement only this stage. First inspect the current repository and the applicable TDD sections. Preserve working code and completed stage behavior. Stay within the listed scope; do not begin later stages or unrelated refactors.

Use current official documentation for the exact dependency versions in the lockfile. Validate all untrusted inputs and keep secrets server-only. Add/adjust focused tests for this stage. Run relevant tests, type-check, lint and build where feasible. Never claim a check passed unless you ran it and report its actual result.

At completion, report: (1) changed files, (2) implementation decisions and assumptions, (3) commands run and pass/fail results, (4) acceptance criteria passed/failed/not tested, (5) risks or owner decisions, (6) next stage only if this gate passes. If blocked, report the minimal blocker and stop without guessing.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 20 - Security, accessibility, performance and resilience audit

Run focused hardening across the completed MVP before deployment.

Objective

Run focused hardening across the completed MVP before deployment.

Preconditions

Read this TDD's applicable product, architecture and template sections before editing.

Confirm previous stages that this stage depends on are accepted; otherwise report the blocker and stop.

Inspect the live repository and fit implementation to actual code rather than assuming the example paths are exact.

Work items - do only these

Audit authentication/session boundaries, server action authorization, draft ownership, upload permissions, file size/type validation and secret exposure.

Check dependency vulnerabilities and review direct dependencies; explain severity and avoid blind major-version upgrades.

Test invalid document payloads, unknown template IDs, malicious filenames, path traversal attempts and unauthorized asset IDs.

Run accessibility checks for contrast, focus order, labels, modal trapping/close, touch target size and reduced motion.

Measure editor/render/export performance on a real mid-range phone; identify large fonts/images and expensive effects.

Handle save/export/upload failures with recoverable messages and no false success states.

Add regression tests for each issue fixed and document any accepted risk.

Files and modules in scope

Security review report, accessibility/performance fixes, regression tests and known-issues list.

Dependency audit outputs and environment/secret checks.

No unrelated template redesign in this stage.

BOUNDARY

Perform a focused quality/security pass only. Fix verified defects, add regression tests, document residual risks and stop. Do not use the audit as permission for unrelated refactors.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 20 - Acceptance and Antigravity prompt

The stage is complete only when its evidence is reported and its gate is passed.

Acceptance criteria

Protected operations enforce server-side identity and ownership.
Upload and input limits are active and documented.
No unresolved critical/high issue is accepted without explicit owner decision.
Keyboard/touch accessibility and core resilience tests pass.
Performance issues are evidenced by measurements, not guesswork.

Verification plan

Run full tests, type-check, lint, build and dependency audit.
Execute unauthorized access and invalid-input scenarios.
Test at minimum one real phone and one desktop browser.
Re-run export regression suite after any renderer/security changes.

STOP CONDITION

Perform a focused quality/security pass only. Fix verified defects, add regression tests, document residual risks and stop. Do not use the audit as permission for unrelated refactors. Do not proceed to the next stage until the owner accepts this report.

Copy-ready Antigravity prompt

COPY-READY / TECHNICAL BLOCK

You are implementing Ozone Lounge Poster Generator TDD Stage 20: Security, accessibility, performance and resilience audit. Implement only this stage. First inspect the current repository and the applicable TDD sections. Preserve working code and completed stage behavior. Stay within the listed scope; do not begin later stages or unrelated refactors.

Use current official documentation for the exact dependency versions in the lockfile. Validate all untrusted inputs and keep secrets server-only. Add/adjust focused tests for this stage. Run relevant tests, type-check, lint and build where feasible. Never claim a check passed unless you ran it and report its actual result.

At completion, report: (1) changed files, (2) implementation decisions and assumptions, (3) commands run and pass/fail results, (4) acceptance criteria passed/failed/not tested, (5) risks or owner decisions, (6) next stage only if this gate passes. If blocked, report the minimal blocker and stop without guessing.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 21 - Deployment, environments and operations

Create a reproducible deployment that behaves like local/staging tests.

Objective

Create a reproducible deployment that behaves like local/staging tests.

Preconditions

Read this TDD's applicable product, architecture and template sections before editing.

Confirm previous stages that this stage depends on are accepted; otherwise report the blocker and stop.

Inspect the live repository and fit implementation to actual code rather than assuming the example paths are exact.

Work items - do only these

Select hosting only after confirming compatibility with server runtime, PostgreSQL connectivity, Prisma deployment requirements, storage and export behavior.

Create separate development/staging/production environment configurations and secret boundaries.

Document safe migration command for deployment and a rollback strategy; avoid destructive migration shortcuts.

Configure storage bucket permissions and allowed CORS origins for the chosen origin(s).

Check static font availability, cache behavior, image access, session cookie settings and production public/base URLs.

Add minimum operational logging for auth failures, upload/save/export errors without sensitive content or secrets.

Document backup/restore for database and the chosen storage metadata/assets; define a smoke-test checklist after deployment.

Files and modules in scope

Deployment config/documentation, environment variable example, migration/seed instructions and smoke-test script/checklist.

Operational log/error policy and backup notes.

Hosting-specific configuration committed only where secrets are absent.

BOUNDARY

Implement only deployment/operations configuration. Do not promote to production or change providers without approval. Report the actual tested host, environment and any feature that still differs from local behavior.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 21 - Acceptance and Antigravity prompt

The stage is complete only when its evidence is reported and its gate is passed.

Acceptance criteria

Clean staging deploy builds from the lockfile and migrations.
Secrets remain in provider settings/server environment.
Login, upload, save/reopen, preview and export pass at the deployed origin.
Backup/restore and rollback instructions are concrete and tested or clearly marked unverified.

Verification plan

Build/deploy staging from a clean revision.
Run smoke tests at the deployed URL and inspect browser/server logs.
Test storage access/CORS and canvas export from the production origin.
Review migrations and secret handling before production promotion.

STOP CONDITION

Implement only deployment/operations configuration. Do not promote to production or change providers without approval. Report the actual tested host, environment and any feature that still differs from local behavior. Do not proceed to the next stage until the owner accepts this report.

Copy-ready Antigravity prompt

COPY-READY / TECHNICAL BLOCK

You are implementing Ozone Lounge Poster Generator TDD Stage 21: Deployment, environments and operations. Implement only this stage. First inspect the current repository and the applicable TDD sections. Preserve working code and completed stage behavior. Stay within the listed scope; do not begin later stages or unrelated refactors.

Use current official documentation for the exact dependency versions in the lockfile. Validate all untrusted inputs and keep secrets server-only. Add/adjust focused tests for this stage. Run relevant tests, type-check, lint and build where feasible. Never claim a check passed unless you ran it and report its actual result.

At completion, report: (1) changed files, (2) implementation decisions and assumptions, (3) commands run and pass/fail results, (4) acceptance criteria passed/failed/not tested, (5) risks or owner decisions, (6) next stage only if this gate passes. If blocked, report the minimal blocker and stop without guessing.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 22 - User acceptance testing, handover and release sign-off

Validate the real operator workflow and leave maintainable handover artifacts.

Objective

Validate the real operator workflow and leave maintainable handover artifacts.

Preconditions

- Read this TDD's applicable product, architecture and template sections before editing.
- Confirm previous stages that this stage depends on are accepted; otherwise report the blocker and stop.
- Inspect the live repository and fit implementation to actual code rather than assuming the example paths are exact.

Work items - do only these

- Prepare UAT cases from the acceptance matrix and source-backed event facts.
- Run the full journey on a real phone: login, choose template, edit, upload/crop, preview, pinch/pan, close, save, reopen and download.
- Run the same functional workflow for all seven templates and capture representative export files.
- Confirm approved logo, font licences, images, event time/location/contact and the publishing aspect ratio; close or explicitly accept every production gate.
- Provide concise operator instructions plus technical notes for adding a template, managing fonts/assets and rotating credentials.
- Record known limitations, browser support and recovery steps; identify any test that could not be run.
- Tag/commit the approved release only after owner sign-off and clean repository state.

Files and modules in scope

- UAT checklist/results, operator quick-start, developer handover, template extension steps and known limitations.
- Release note and sign-off/rollback record.
- Final screenshots/export samples may be stored in agreed project documentation.

BOUNDARY

Perform UAT and handover only. Do not add new features during sign-off. Report passed/failed/not-tested for every acceptance case and stop for owner release approval.

IV. ANTIGRAVITY EXECUTION STAGES

STAGE 22 - Acceptance and Antigravity prompt

The stage is complete only when its evidence is reported and its gate is passed.

Acceptance criteria

All acceptance criteria have evidence; no unverified claim is marked passed.
Owner approves visual identity, source copy and final output aspect ratio.
All seven templates save/reopen and export at the approved dimensions.
Outstanding issues have named disposition: fix, accepted risk or deferred enhancement.
Repository is clean and build/test commands are documented.

Verification plan

Run acceptance suite on desktop and real mobile device.
Inspect all seven final exports and compare them to live preview.
Confirm backups, deployment rollback and account provisioning.
Obtain explicit product-owner release sign-off.

STOP CONDITION

Perform UAT and handover only. Do not add new features during sign-off. Report passed/failed/not-tested for every acceptance case and stop for owner release approval. Do not proceed to the next stage until the owner accepts this report.

Copy-ready Antigravity prompt

COPY-READY / TECHNICAL BLOCK

You are implementing Ozone Lounge Poster Generator TDD Stage 22: User acceptance testing, handover and release sign-off. Implement only this stage. First inspect the current repository and the applicable TDD sections. Preserve working code and completed stage behavior. Stay within the listed scope; do not begin later stages or unrelated refactors.

Use current official documentation for the exact dependency versions in the lockfile. Validate all untrusted inputs and keep secrets server-only. Add/adjust focused tests for this stage. Run relevant tests, type-check, lint and build where feasible. Never claim a check passed unless you ran it and report its actual result.

At completion, report: (1) changed files, (2) implementation decisions and assumptions, (3) commands run and pass/fail results, (4) acceptance criteria passed/failed/not tested, (5) risks or owner decisions, (6) next stage only if this gate passes. If blocked, report the minimal blocker and stop without guessing.

V. VERIFICATION AND OPERATIONS

Deployment architecture options

Choose hosting after the auth, database and export paths have been tested.

Requirements

The selected host must support the chosen Next.js App Router/server execution model, reliable PostgreSQL connectivity, the installed Prisma configuration, static font/logo files and the selected object storage. Storage CORS must match the deployed application origin. Staging should use a separate data boundary from production.

Deployment principles

Keep environment config out of source files.

Use explicit, reviewed database migrations.

Pin runtime/package versions and commit the lockfile.

Test fonts, upload, save/reopen and export at the deployed origin.

Do not choose a provider based only on a successful desktop build; browser exports and database connections need runtime testing.

V. VERIFICATION AND OPERATIONS

Environment variables and secret management

This document lists categories and example names only; never place live secrets here.

Configuration categories

Database connection; authentication secret/base URL; object storage URL/bucket and server-only key; public app URL; upload size/type limits; poster width/height; optional logging keys.

Secret rules

Private credentials must not use public client prefixes.
Never print secret values in logs or error messages.
Keep .env.example with placeholders only and real local files ignored by Git.
Staging and production use separate secrets and data scopes.
If a secret is committed accidentally, rotate it; deleting the line in a later commit is insufficient.

COPY-READY / TECHNICAL BLOCK

```
DATABASE_URL=replace-me
AUTH_SECRET=replace-with-random-secret
APP_BASE_URL=http://localhost:3000
STORAGE_URL=replace-me
STORAGE_BUCKET=ozone-poster-assets
STORAGE_SERVICE_KEY=server-only-secret
POSTER_WIDTH=1080
POSTER_HEIGHT=1350
```

V. VERIFICATION AND OPERATIONS

Backup, recovery and asset retention

Drafts and source assets are operational content and need a recovery plan.

Database recovery

Document backup frequency, retention, restore owner and restoration procedure. Keep schema migrations and seed definitions under version control. Test a restore to non-production before release. Do not assume backups exist unless the selected provider/plan confirms them.

Asset retention

- Define how old unreferenced uploads are discovered and cleaned.
- Never run destructive cleanup until asset references are validated.
- Do not delete an image still referenced by another draft.
- Decide what happens to draft/asset data when an operator is disabled or a draft is deleted.
- If the client wants to archive finished artwork, define whether exported binaries are saved separately from editable drafts.

V. VERIFICATION AND OPERATIONS

Monitoring and operational logs

Observe failures without recording secrets or unnecessary poster content.

Useful signals

Error rate for draft load/save/delete and asset upload/resolve.
Export failures grouped by font, image/CORS, timeout or generic rasterisation category.
Auth failure and rate-limit counts.
Slow database operations/connection issues.
Deployment and migration failures.

Log hygiene

Never log passwords, cookies, tokens, storage service keys or entire environment variables. Avoid logging full poster copy or private asset URLs unless necessary for a diagnosed issue. Use request/correlation IDs to relate user-visible errors to server logs. Define log access and retention. Keep rollback instructions available for a bad release.

V. VERIFICATION AND OPERATIONS

Risk register and mitigation plan

Validate the expensive risks early so template implementation does not multiply rework.

Risk	Mitigation / gate
Premium font fails in export	Stage 6 font proof and Stage 12 real-font export tests.
Mobile export is unreliable	Stage 7 spike, then real iOS Safari/Android Chrome tests.
Templates look like recolours	Unique renderers and a separate creative acceptance checklist.
Local object URL is saved	Reject blob references in server validation; save stable asset IDs.
Username auth needs internal email	Stage 3 proof of concept and explicit provisioning decision.
Missing time/venue/contact	Preflight warning; do not invent data.
Blank Friday offer breaks layout	Conditional offer component and no-offer visual fixture.
Template schema changes break drafts	Version both document and template; test migration/compatibility.

Keep each risk owner and disposition in the project decision log. “Not tested” is not the same as “passed.”

V. VERIFICATION AND OPERATIONS

Definition of done for every stage

A stage is complete only when implementation, verification and handoff are all complete.

Implementation done

Changes stay within the authorised stage; no unrelated repo-wide rewrite.
Types/contracts and error states match the touched feature.
Responsive behavior is checked at phone size.
No placeholder button pretends to work.
No secrets or invented event facts are introduced.

Verification done

Run applicable lint, typecheck, build, unit/component/E2E checks.
Mark each acceptance criterion PASS, FAIL or NOT TESTED.
Report the real command and result, including failures.
Manual checks include viewport/browser details.

Handoff done

List changed files, migrations/env actions, unresolved decisions, limitations, next-stage prerequisites and a Git checkpoint/commit where repository practice allows.

V. VERIFICATION AND OPERATIONS

Antigravity operating protocol

These guardrails address common coding-agent failure modes: broad rewrites, context drift and unverified completion.

Before coding

Inspect the actual route tree, package scripts, dependencies, environment examples, tests and Git status.
Identify existing user work and preserve uncommitted changes.
Read the stage prompt and relevant TDD sections.
Give a brief implementation plan before modifying files.
If the repo differs from the TDD, describe the difference and propose the smallest safe fix.

While coding

Make small, reviewable changes.
Do not delete working features outside the stage.
Do not silently change package manager, database/auth provider or canvas schema.
Do not add duplicate dependencies without reason.
If blocked, report the blocker and complete only safe work.

After coding

Run checks, report results, list files, keep known limitations visible and stop. Do not implement the next stage in the same agent run.

V. VERIFICATION AND OPERATIONS

Reusable Antigravity prompt skeleton

Use this prompt structure for any approved maintenance task as well as the formal stages.

COPY-READY / TECHNICAL BLOCK

```
You are implementing Stage [N] of the Ozone Lounge Poster Generator TDD v1.0.
Inspect the repository first; do not assume it is empty.
Implement only this stage and preserve all working behavior.
Do not implement later stages or unrelated refactors.
Follow the stage acceptance criteria and run the required checks.
If blocked, describe the exact blocker instead of inventing a workaround.
Report files changed, commands/results, failed or untested criteria, risks and decisions.
Stop after the stage report.
```

A bounded prompt lets Antigravity focus on one verifiable capability, while the handoff report preserves architectural decisions between sessions. Checkpoint the repo between stages so a risky renderer/export experiment can be rolled back without losing known-good work.

V. VERIFICATION AND OPERATIONS

Stage gate map and dependency order

The sequence validates high-risk infrastructure before multiplying design work.

Gate	Must exist before passing
G1 Foundation	App boots; lint/typecheck/build baseline is captured.
G2 Authentication/data	Protected access and persistent draft-service smoke test pass.
G3 Renderer/export	Fixed canvas plus supplied-font/image export proof passes.
G4 Vertical slice	Reddington is editable, previewable, saveable and downloadable.
G5 Template coverage	Six remaining unique renderers pass their fixture/export checks.
G6 Release	Security, mobile QA, deployment checks and owner signoff pass.

Reddington is first because it exercises the core workflow end-to-end. Later templates exercise distinct layout challenges: geometry (Saint), neon/offer hierarchy (Lannister), role separation (Shelby), no offer (Lothbrok), colourful layers (Escobar) and texture/motifs (Pinkman). Do not call later templates complete if shared rendering/export remains unstable.

V. VERIFICATION AND OPERATIONS

Source-of-truth content matrix

Use the supplied Ozone Lounge Weekly Entertainment & Promotions Plan as the content fixture source.

Day	Template	Event / lineup	Offer from source brief
Monday	Reddington	Old Classics & Sundowners; Monsieur Pierre	Discounts on all drinks, excluding wine.
Tuesday	Saint	African Night (Afro); DJ Neb & DVJ Jojez	Discounts on beer buckets and cocktails.
Wednesday	Lannister	Groovy Wednesday; DJ Donben & DJ Vin	Free koroga chicken on the house; discounts on whiskey bottles, cognac and beer buckets.
Thursday	Shelby	Karaoke Night; Hosts Bellah Harriet & Tito Wagithomo; DJ Fab	Discounts on gin, rum and vodka; free tequila shots for singers.
Friday	Lothbrok	Flight Fridays; DJ Slim D & Mistabrand	No offer was supplied. Persist blank fields, not placeholder copy.
Saturday	Escobar	Saturday Fiesta; DVJ Jojez	Discounts on whiskey bottles and cognac.
Sunday	Pinkman	Old Skool Sundays; DJ Ngash	Discounts on beer buckets.

Regression tests should assert these values and role assignments. If the client later changes copy, update the source fixture deliberately rather than letting a renderer silently drift.

V. VERIFICATION AND OPERATIONS

Example document contract

Illustrative only; actual TypeScript definitions must match the final editor and schema validators.

COPY-READY / TECHNICAL BLOCK

```
interface PosterDocument {
  schemaVersion: number;
  templateId: TemplateId;
  templateVersion: number;
  name: string;
  canvas: { width: number; height: number; format: string };
  content: {
    eventTitle: string; dateText: string; startTime: string;
    venueName: string; locationText: string; contactText: string;
    socialHandle: string; primaryOffer: string; secondaryOffer: string;
    djNames: string[]; hostNames: string[]; customLines: string[];
  };
  assets: { logoAssetId: string | null; backgroundAssetId: string | null; secondaryAssetIds: string[] };
  styling: { paletteId: string; typographyPresetId: string; layoutOptionId: string;
    decorationOptions: Record<string, string | number | boolean>;
    focalPoints: Record<string, { x: number; y: number }> };
}
```

Keep stored JSON small, serialisable and validated. Do not save temporary File objects, DOM nodes, zoom/pan or application theme in this object.

V. VERIFICATION AND OPERATIONS

Environment and release checklist

List variable names in documentation; secret values belong only in deployment settings.

Configuration checklist

Database URL and pooling mode match the selected hosting/docs.
Authentication secret/base URL are configured in the server environment.
Object storage and bucket/path permissions are configured.
CORS allows only approved deployed origins required by upload/export.
Canvas width/height match the approved publishing channel.
Upload file-size and MIME-type rules are configured.
Staging and production use separate secrets and data boundaries.

Release checklist

Migration/build pass; no secrets in Git or client bundle.
Login and every protected data operation are tested.
All seven defaults render and export correctly.
Logo, licensed fonts and approved images are ready.
Date/time/venue/contact decisions are closed or consciously omitted.
Backup/restore and rollback process is documented.
Owner has reviewed remaining limitations.

V. VERIFICATION AND OPERATIONS

Final acceptance test matrix

Store completed evidence with the release so “done” is reproducible.

ID	Acceptance check
ACC-01	Seven registry entries appear in weekday order with correct tags.
ACC-02	Authentication protects pages and draft/asset operations.
ACC-03	A saved draft reopens with stable assets and styles after relogin.
ACC-04	Preview zoom/pan does not alter canvas geometry or export dimensions.
ACC-05	PNG/JPEG dimensions match configured master dimensions.
ACC-06	Mobile save/export flow passes target browser tests.
ACC-07	Missing logistics trigger warnings without invented content.
ACC-08	Reddington and all six later templates have unique compositions.
ACC-09	Blank Friday offer and Shelby's role separation are correct.
ACC-10	Long text, missing image, failed save and failed export recover gracefully.

Owner walkthrough: login -> choose template -> edit -> preview/zoom -> close -> save -> reopen -> download. Repeat for every template and at least one real phone.

V. VERIFICATION AND OPERATIONS

Known limitations and change control

Record untested behavior honestly and distinguish it from accepted scope.

MVP limitations

No direct social publishing/scheduling, no unrestricted freeform object editor, no complex offline collaboration, and one approved master output format until the owner confirms the channel. Provider-specific mobile download behavior may vary.

Change classification

Bug: breaks an accepted criterion; fix and add regression coverage.

Decision: resolves a documented unknown; update the decision register and affected stages.

Enhancement: adds out-of-scope functionality; create a separate stage.

Architecture change: changes auth, storage, database, renderer or canvas contract; provide impact analysis and obtain owner approval.

Never mix unrelated enhancements into a bug-fix stage. Keep the implementation log and source facts synchronized with any approved change.

APPENDIX F. REFERENCES

Source documents and technical references

Product facts come from the user-provided project inputs. Technical references support implementation choices.

Project inputs

POSTER_GENERATOR_ARCHITECTURE.md - existing Lucky House rendering, scaling, font injection, image lifecycle, export and draft persistence patterns.

ozone_lounge_weekly_posters_outline.pdf - Ozone event titles, lineups, promotional wording, creative direction and final production checklist.

Ozone_Lounge_Poster_Generator_Blueprint.md - Ozone product plan, mobile-first UX, template catalogue, typed document model and stage plan.

Official technical references checked 18 October 2026

Next.js App Router docs: https://nextjs.org/docs/app

Better Auth username plugin: https://better-auth.com/docs/plugins/username

Better Auth email/password: https://better-auth.com/docs/authentication/email-password

Prisma ORM with Next.js: https://www.prisma.sh/docs/guides/frameworks/nextjs

Prisma PostgreSQL quickstart/release status: https://www.prisma.io/docs/prisma-orm/quickstart/postgresql

OWASP Password Storage Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

MDN CORS-enabled images and canvas restrictions: https://developer.mozilla.org/en-US/docs/Web/HTML/How_to/CORS_enabled_image

Supabase Storage access control, if Supabase is chosen: https://supabase.com/docs/guides/storage/security/access-control

MDN HTMLCanvasElement.toBlob(): https://developer.mozilla.org/en-US/docs/Web/API/HTMLCanvasElement/toBlob

Versions and API signatures change. The agent must check docs matching the version pinned in package.json, not blindly copy a setup from an older major version.

APPENDIX G. BUILD CONTROL

Agent execution report template

Require this report at the end of every Antigravity stage.

Required report

Stage number and name; date; branch/commit if available.
Summary of changes in plain language.
Files added, changed and deleted, with reason for each deletion.
Commands actually executed and exact outcomes.
Acceptance criteria marked PASS, FAIL or NOT TESTED with evidence.
Known defects, security concerns, performance concerns and unresolved decisions.
Confirmation that no later stage or unrelated refactor was started.
Recommended next stage only if the current gate is accepted.

GATE DISCIPLINE

“Code written” is not a pass. The stage is passed only when the tests and observable acceptance criteria pass, or the owner explicitly accepts an identified limitation.

APPENDIX H. PRODUCTION DECISION REGISTER

Decisions that must not be silently guessed

Use this table as a living register during Stage 0 and release preparation.

Decision	Safe interim behavior	Production exit requirement
Primary publishing channel/aspect ratio	Configurable master canvas; provisional 1080 x 1350 only for feed hypothesis	Owner selects feed 4:5 or Story/Status 9:16 (or other)
Official logo	Placeholder labelled development-only	Approved original logo and any permitted variants supplied
Hero/DJ imagery	Licensed placeholder/sample imagery only	Approved images or documented licence/provenance
Event start time/date	Blank optional field	Owner confirms whether each is required
Venue/location/contact/social	Blank optional fields; conditional rendering	Owner confirms correct publishing fields
Friday special offer	Blank, no visible offer module by default	Leave blank or provide authorised wording
Authentication identity	Username is entered at login	Confirm secure provisioning path and whether internal email metadata is allowed
Draft visibility	Private, owner-scoped drafts	Owner confirms whether team-shared drafts are required
Export formats	PNG and JPEG target	Confirm operational needs and verify target-device behavior
Storage/provider	Abstract adapter until approved	Select provider and validate permissions, CORS, costs and backup

Do not convert an interim default into a permanent product decision without owner approval.

APPENDIX I. EDGE-CASE TEST MATRIX

Content, assets and export edge cases

Use these fixtures across the seven renderers and after shared renderer changes.

Area	Fixture	Expected behavior
Title	Very long event title	Wrap/scale according to design rule; never clip silently
Lineup	Multiple names or long host names	Clear wrapping; preserve names and roles
Offer	Empty Friday offer	Hide offer element and rebalance layout
Offer	Long multi-part Wednesday offer	Preserve every category; deliberate wrapping/grouping
Optional field	Missing venue, time or social	Hide blank line; preflight can warn if required
Font	Selected TTF/OTF fails to load	Visible fallback warning; export remains usable
Image	Missing, corrupt or too-large image	Fallback/warning and recoverable upload error
Image	Portrait vs landscape source photo	Predictable crop with saved position; no stretched faces
Asset URL	Temporary blob URL after reload	Must not be considered durable; recover from stored asset
Export	Font not ready	Wait or fail clearly; do not export fallback unnoticed
Export	CORS/tainted canvas failure	Explain failure; no false success
Mobile	Pinch and pan before export	Export stays identical at master dimensions
Save	Network loss during save	Preserve local changes and show retry state
Access	User changes draft ID to another account's ID	Server rejects unauthorized operation
Compatibility	Older schemaVersion draft	Migrate or show controlled unsupported-draft error
Theme	Switch app dark/light	Poster appearance remains unchanged

Every fixed defect adds a regression test. Keep test fixture imagery licensed or generated for testing.

APPENDIX J. TEMPLATE EXTENSION WORKFLOW

Adding template eight and beyond

New templates extend the registry and renderer contract rather than duplicating the product.

Required extension package

Stable template ID, human name, day or category, best-for tag and gallery thumbnail.
Versioned default poster document with validated content and assets.
Unique composition rules and design tokens; do not define uniqueness as a palette swap alone.
Curated option schema for palettes, fonts, crop, decorations and conditional content.
Renderer that accepts the common poster document and respects configured canvas dimensions.
Content fixtures, empty/missing-field fixtures, long-copy fixtures, save/reopen tests and export QA.
Licence/provenance for font files and imagery.

Required tests before registration is merged

Registry ID/name uniqueness and weekday/category ordering behavior.
Default data validates and matches approved source copy.
All options save and reopen without loss.
Rendered output fits the master canvas and works at small screen preview size.
Export file dimensions and representative pixel/layout checks pass.
Existing templates do not regress.

Do not introduce per-template custom database tables unless a documented capability truly requires them.

APPENDIX K. SECURITY AND OPERATIONS QUICK REFERENCE

Secure handling rules for the implementation agent

Treat security as a system property spanning server actions, database, storage and browser code.

- Authentication library manages password hashing and session lifecycle; never store plaintext passwords or write a new crypto scheme.
- Authorization is checked for each protected read/write/delete/export-adjacent storage action; hidden buttons are not access control.
- Keep provider service keys, database URLs and auth secrets in server-side environment variables only.
- Validate file size, actual content type/signature where feasible, and image decode before storing. Use random storage object keys, not raw untrusted filenames.
- Store media metadata and poster documents as separate concerns. Only approved users may reference private stored assets.
- Log request IDs and error categories, not passwords, auth cookies, secret values or unnecessary poster content.
- Configure rate limits for login and upload paths, conservative request-body limits and secure cookie defaults through the chosen library/platform.
- Apply migrations deliberately. Back up database and media references; test restoration and document deletion retention.
- Revisit dependencies and official docs when version pins change. Do not upgrade major dependencies during unrelated template work.

CRITICAL RISK

A private draft document can reference an image URL directly. Ensure the storage access model matches the approved user/draft visibility policy rather than relying only on the obscurity of a URL.

APPENDIX L. RELEASE CHECKLIST

Owner sign-off and go-live checklist

Treat this as the final release gate after Stage 22.

Product and content

[ ] Final output aspect ratio and master dimensions are approved.
[ ] Official logo, images and font licences are approved and inventoried.
[ ] Event names, lineups and promotions match the source brief.
[ ] Start times, date, venue, contact and social details are confirmed or deliberately omitted.
[ ] Friday's offer policy is confirmed; no offer is fabricated.

Engineering and operations

[ ] Seven templates pass render/save/reopen/preview/export acceptance tests.
[ ] Authentication, authorization, upload and draft-ownership tests pass.
[ ] Mobile browser export is verified on the target device(s).
[ ] Production build and migrations are verified from the release revision.
[ ] Secrets, storage permissions, CORS, backups and rollback are checked.
[ ] Known limitations and untested items are disclosed.
[ ] Operator and developer handover instructions are delivered.
[ ] Owner explicitly approves the production release.

Release approver: _____ Date: _____